

CV Lab Mid-Exam Cheat Sheet

AI-4002 Computer Vision Lab · Labs 01–06 · Revised Oct 7, 2026

0. Exam brief (read this first)

The exam is open book, closed internet, 3 questions, Q1 is MCQs, and it covers Lab Manuals 1 to 6, including the Lab 2 photometric topics that were never uploaded.

Sir Talha's announcement (GCR, Oct 4)

Item	Rule	What it means for you
Format	Open book, strictly closed internet	No Stack Overflow, no docs site. Everything must be printed or in your head.
Allowed materials	Printed cheat sheets for Python and OpenCV only	Print this. Signatures (Section 9) matter more than prose.
Layout	3 questions; Q1 = MCQs	Q1 tests theory and function behaviour. Q2 and Q3 are almost certainly coding tasks.
Syllabus	Lab Manual 1 through Lab Manual 6	Labs 1, 3, 4, 5, 6 manuals plus all task sheets.
Lab Manual 2	Not on GCR, still examinable	Photometric / point transforms: Log , Gamma , Piecewise , Histogram (Section 4).
Detailed breakdown	Promised "later tonight or tomorrow morning"	Not in the shared files. If it arrives, check it against Section 9.

How every code block in this sheet is built

- **Run Section 1.4 once** at the top of your notebook. It defines the imports, the `output/` folder, `load()`, `show()` and the size/channel helpers that every later block uses.
- **Every block starts** with `import cv2, numpy as np, matplotlib.pyplot as plt`, so the imports are never missing even if you paste one block alone.
- **Every image is loaded with** `load(path)`, which raises a clear error when the path is wrong. No block nests `cv2.imread` inside another call.
- **Every task block ends with** `show(...)` and `cv2.imwrite("output/...")`, because Lab 5 requires both a visual comparison and saved results. Pure-Python tasks (Lab 1, Tasks 1 and 2) have no image, so they print instead.
- **Every display goes through** `show()`, which converts BGR to RGB. Titles use f-strings that print the parameters, for example `f"Canny low={lo}, high={hi}"`.
- **File names** are placeholders under `data/`. Change only the string.

Six habits that save marks

- Check the path: `load()` does it for you. Raw `cv2.imread` returns `None` silently on a wrong path.
- Convert BGR to RGB before `plt.imshow`, or reds and blues swap. `show()` does this.
- Single-channel images need `cmap="gray"` in Matplotlib. `show()` does this.
- Do maths in `float32`, then `np.clip(..., 0, 255).astype(np.uint8)`. `uint8` arithmetic wraps: $250 + 10 = 4$.
- OpenCV sizes are (w, h) ; NumPy shapes are (h, w) . This is the number-one source of bugs.
- Put parameter values in every subplot title. Graders read titles first.

Errors in the manuals (likely MCQ traps)

Where	What the manual says	What is actually true
Lab 3, p.3 list	Translation listed under perspective transformation	Translation is affine . The manual's own diagram on p.4 shows this correctly.

Where	What the manual says	What is actually true
Lab 1, 11.9 text	Threshold 100, maxval 200 "represents pure white"	200 is light grey. Pure white is 255.
Lab 1, 11.1 description	Says <code>cv2.imshow</code> and <code>waitKey(0)</code> are used	The code uses <code>plt.imshow</code> . <code>cv2.imshow</code> does not display inline in Jupyter.
Lab 1, 11.5 comment	"(5, 5) is the kernel size"	The code passes <code>(31, 31)</code> . Kernel sides must be odd.
Lab 4, 4.2.1.2	<code>BORDER_CONSTANT</code> gives "valid" (no padding)	<code>filter2D</code> always returns the input size. True valid output means cropping $\lfloor k/2 \rfloor$ pixels per side.
Lab 4, 4.2.1.3	"Zero padding" using <code>BORDER_REFLECT</code>	<code>BORDER_REFLECT</code> mirrors pixels. Zero padding is <code>BORDER_CONSTANT</code> .
Lab 4, 4.2.1.4	<code>cv2.filter2D(..., strides=(2,2))</code>	<code>filter2D</code> has no <code>strides</code> argument. Stride 2 means slicing the output <code>::2, ::2</code> .
Lab 4, 3.6 code	<code>contrast = filter2D(image, ones)</code>	That is a local sum . Contrast is the local standard deviation $\sqrt{\mathbb{E}[I^2] - \mathbb{E}[I]^2}$.
Lab 4, 3.6 code	<code>image ** 2</code> on uint8	Overflows. Cast to <code>float32</code> first.
Lab 4, 3.4 code	<code>arctan2</code> angles binned over $[0, 360)$	<code>arctan2</code> returns $(-180^\circ, 180^\circ]$. Apply <code>% 360</code> first.
Lab 4, 4.1 code	<code>getGaussianKernel(5, 1)</code> passed straight to <code>filter2D</code>	That kernel is 5×1 , so it blurs vertically only. Use $g g^T$.
Lab 4, 3.6 text	"Higher energy = less uniform texture"	For GLCM energy the opposite holds. For the manual's local sum of squares, high just means bright. Say which definition you use.
Lab 4 HED	HED = Histogram of Edge Directions	Not Holistically-Nested Edge Detection (a deep network).
Lab 5, 2.3.4 code	<code>image[markers == -1] = [255,0,0]</code> called "red"	Images are BGR, so <code>[255,0,0]</code> is blue . Red is <code>[0,0,255]</code> .
Lab 5, 2.3.3 code	Seed <code>(10, 10)</code> indexed as <code>image[x, y]</code>	NumPy indexing is <code>[row, col] = [y, x]</code> . It works only because it is consistent.
Lab 1, 11.10	"positive angle = counter-clockwise"	Correct on screen, because the y axis points down.

1. Environment, repository & master utilities

Install packages before the exam (the internet is closed during it), work in one VS Code Jupyter notebook, and run the master utility block in Section 1.4 as your first cell.

1.1 VS Code + Jupyter in 6 steps (do this tonight)

1. Install Python 3.10 to 3.12 and VS Code. In VS Code install the **Python** and **Jupyter** extensions (Microsoft).
2. Open your exam folder: File → Open Folder.
3. Create a virtual environment in the terminal:
 1. Windows: `python -m venv .venv` then `.venv\Scripts\activate`
 2. macOS/Linux: `python3 -m venv .venv` then `source .venv/bin/activate`
4. Install everything now: `pip install opencv-contrib-python numpy matplotlib pandas scikit-image PyWavelets ipykernel ipywidgets`
5. Ctrl+Shift+P → "Create: New Jupyter Notebook". Top right → Select Kernel → choose `.venv`.
6. Run the check cell below. If it prints versions and `SIFT ok`, you are ready.

Install only one of `opencv-python` and `opencv-contrib-python`; having both causes import clashes. `cv2.SIFT_create()` exists in the main package since OpenCV 4.4.

```
# Environment check (Lab 1, Part 1.2 "Import & Validate")
import sys, cv2, numpy as np, matplotlib, pandas as pd
print("Python      :", sys.version.split()[0])
print("OpenCV      :", cv2.__version__)
print("NumPy       :", np.__version__)
```

```
print("Matplotlib:", matplotlib.__version__)
print("Pandas      :", pd.__version__)
test = np.zeros((50, 50, 3), np.uint8)
assert cv2.cvtColor(test, cv2.COLOR_BGR2GRAY).shape == (50, 50)
cv2.SIFT_create()
print("SIFT ok")
```

Inside Jupyter, install with `%pip install opencv-python` (Lab 1, Part 1.1). `%pip` targets the notebook's own kernel; `!pip` may install somewhere else.

1.2 Repository layout (Lab 2 submission rules)

```
StudentName_Medical_Imaging_Portfolio/
├── Task_1_Chest_XRay/
│   ├── data/           # only the X-ray images you used
│   ├── output/         # saved enhanced results
│   ├── xray_enhancement.ipynb
│   └── README.md       # pipeline explanation
├── Task_2_Cardiac_Fusion/
│   ├── data/ output/ modal_fusion.ipynb README.md
├── Task_3_Echo_Analysis/
│   ├── data/ output/ realtime_echo.ipynb README.md
├── requirements.txt     # global dependencies
└── README.md           # overview + setup instructions
```

- **100 MB rule:** GitHub rejects large files. Copy only the sample images or short clips you use into `data/`; never upload whole datasets.
- **Relative paths only:** `load("data/sample_xray.png")`, never `C:/Users/...`.
- **requirements.txt** pins versions so a grader can rebuild your environment. Generate it with `pip freeze > requirements.txt`, then delete unused lines.

```
opencv-python==4.8.0.76
numpy==1.26.0
matplotlib==3.8.0
pandas==2.1.1
scikit-image==0.22.0
PyWavelets==1.5.0
```

1.3 Path rules

- `cv2.imread` never raises an error; it returns `None` on a wrong path, a wrong extension, or (on Windows) non-English characters in the path. `load()` below turns that into a clear message.
- Check where Python is running: `print(os.getcwd())`. VS Code runs a notebook from the notebook's own folder.
- `cv2.imwrite` expects BGR. Save the BGR image, not the RGB copy you made for plotting.
- `cv2.imwrite` picks the format from the extension: `.png` is lossless, `.jpg` is lossy.

1.4 Master utility block (run first, every later block depends on it)

```
# ===== MASTER UTILITIES: run this cell first =====
import os, cv2, numpy as np, matplotlib.pyplot as plt
from pathlib import Path

os.makedirs("output", exist_ok=True) # every block saves into output/

def load(path, flag=cv2.IMREAD_COLOR):
    """Load an image or stop with a clear error. flag: IMREAD_COLOR / IMREAD_GRAYSCALE / IMREAD_UNCHANGED."""
    img = cv2.imread(str(path), flag)
    if img is None:
        # Non-ASCII path fallback (Windows), then a clear error
        try:
            img = cv2.imdecode(np.fromfile(str(path), np.uint8), flag)
        except (FileNotFoundError, OSError):
            img = None
    if img is None:
```

```

        raise FileNotFoundError(f"Error: could not load image at {Path(path).resolve()}")
    return img

def show(images, titles=None, cols=None, size=4, cmap="gray"):
    """Plot images side by side. 3-channel images are treated as BGR and converted to RGB."""
    n = len(images)
    cols = cols or n
    rows = int(np.ceil(n / cols))
    plt.figure(figsize=(size * cols, size * rows))
    for i, im in enumerate(images):
        plt.subplot(rows, cols, i + 1)
        if im.ndim == 3:
            plt.imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB))
        else:
            plt.imshow(im, cmap=cmap)
        if titles:
            plt.title(titles[i], fontsize=10)
        plt.axis("off")
    plt.tight_layout()
    plt.show()

def to_bgr(img):
    """1-channel -> 3-channel; 4-channel (PNG with alpha) -> 3-channel."""
    if img.ndim == 2:
        return cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
    if img.shape[2] == 4:
        return cv2.cvtColor(img, cv2.COLOR_BGRA2BGR)
    return img

def match(a, b, mode="resize"):
    """Return b with the same size, channel count and dtype as a. mode: 'resize' stretches, 'pad'
    letterboxes."""
    if a.ndim == 3:
        b = to_bgr(b)
    elif b.ndim == 3:
        b = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY)
    h, w = a.shape[:2]
    if mode == "resize":
        b = cv2.resize(b, (w, h), interpolation=cv2.INTER_AREA) # dsize is (WIDTH, HEIGHT)
    else:
        s = min(w / b.shape[1], h / b.shape[0])
        b = cv2.resize(b, (int(b.shape[1] * s), int(b.shape[0] * s)))
        top = (h - b.shape[0]) // 2
        left = (w - b.shape[1]) // 2
        b = cv2.copyMakeBorder(b, top, h - b.shape[0] - top, left, w - b.shape[1] - left,
                               cv2.BORDER_CONSTANT, value=0)
    return b.astype(a.dtype)

def label(img, text, org=(10, 30), scale=0.8, fg=(255, 255, 255), bg=(0, 0, 0), th=2):
    """putText on a solid background box so it is readable on any image. org = bottom-left of text."""
    (tw, tht), base = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX, scale, th)
    x, y = org
    cv2.rectangle(img, (x - 4, y - tht - 6), (x + tw + 4, y + base + 2), bg, -1)
    cv2.putText(img, text, (x, y), cv2.FONT_HERSHEY_SIMPLEX, scale, fg, th, cv2.LINE_AA)
    return img

def gamma_correct(img, gamma):
    """Power-law  $s = r^{\gamma}$  on normalised intensities, applied as a 256-entry lookup table."""
    table = np.array([((i / 255.0) ** gamma) * 255 for i in range(256)]).astype(np.uint8)
    return cv2.LUT(img, table)

def gray_world(img):
    """Colour balance: scale each BGR channel so its mean equals the overall mean."""
    f = img.astype(np.float32)
    means = f.reshape(-1, 3).mean(axis=0)
    f *= means.mean() / (means + 1e-6)
    return np.clip(f, 0, 255).astype(np.uint8)

print("Master utilities ready. Output folder:", os.path.abspath("output"))

```

1.5 No image on hand? Make one and save it to `data/`

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
os.makedirs("data", exist_ok=True)
img = np.full((400, 600, 3), 255, np.uint8) # white canvas
cv2.rectangle(img, (50, 50), (250, 200), (0, 0, 255), -1) # filled red box (BGR)
cv2.circle(img, (420, 250), 90, (255, 0, 0), -1) # filled blue circle
cv2.putText(img, "TEST", (60, 330), cv2.FONT_HERSHEY_SIMPLEX, 2, (0, 0, 0), 3)
cv2.imwrite("data/sample.jpg", img)
img = load("data/sample.jpg") # always reload through load() to prove the path works
show([img], [f"Synthetic test image {img.shape[1]}x{img.shape[0]}"])
cv2.imwrite("output/synthetic_check.png", img)
```

2. Exam curveballs & reusable patterns

These patterns solve the twists examiners add to a task: unequal image sizes, odd-shaped ROIs, live parameter tuning, blob measurements, folders of images and video.

2.1 Blending two images of unequal size or channel count

`cv2.add`, `cv2.addWeighted`, `cv2.bitwise_*` and `np.hstack` all need the **same height, width, channel count and dtype**, or OpenCV raises `Sizes of input arguments do not match`. Fix the second image with `match()` from Section 1.4.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show(), match() from Section 1.4
img1 = load("data/photo1.jpg")
img2 = load("data/photo2.png", cv2.IMREAD_UNCHANGED) # may be gray or have an alpha channel
alpha = 0.6
fixed_stretch = match(img1, img2, mode="resize") # stretch to img1's size
fixed_pad = match(img1, img2, mode="pad") # keep aspect ratio, black borders
blend_stretch = cv2.addWeighted(img1, alpha, fixed_stretch, 1 - alpha, 0)
blend_pad = cv2.addWeighted(img1, alpha, fixed_pad, 1 - alpha, 0)
show([img1, to_bgr(img2), blend_stretch, blend_pad],
     [f"img1", f"img2 {img2.shape}", f"Stretched, alpha={alpha}", f"Padded, alpha={alpha}"])
cv2.imwrite("output/blend_stretch.png", blend_stretch)
cv2.imwrite("output/blend_pad.png", blend_pad)
```

The blend formula:

$$\text{dst}(x, y) = \alpha \text{src}_1(x, y) + \beta \text{src}_2(x, y) + \gamma$$

- **Interpolation:** `INTER_AREA` for shrinking, `INTER_LINEAR` (default) or `INTER_CUBIC` for enlarging, `INTER_NEAREST` for masks and label images (keeps values exactly 0 or 255).
- **`cv2.add` vs `+`:** OpenCV saturates ($250 + 10 \rightarrow 255$); NumPy uint8 wraps ($250 + 10 \rightarrow 4$).

2.2 ROI coordinate maths

Slicing is always `img[y1:y2, x1:x2]` (rows first). A slice is a **view**: editing it edits the original, so use `.copy()` to keep them separate. For an image of width w and height h , the centre is $(c_x, c_y) = (\lfloor w/2 \rfloor, \lfloor h/2 \rfloor)$.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/sample.jpg")
h, w = img.shape[:2]
cx, cy = w // 2, h // 2

# Fixed-size centre ROI, clamped so it never leaves the image
rw, rh = 300, 300
x1, y1 = max((w - rw) // 2, 0), max((h - rh) // 2, 0)
center_roi = img[y1:y1 + rh, x1:x1 + rw].copy()

# Central N% of the image
N = 50
nw, nh = int(w * N / 100), int(h * N / 100)
```

```

center_pct = img[(h - nh) // 2:(h + nh) // 2, (w - nw) // 2:(w + nw) // 2].copy()

# Quadrants and strips
top_right = img[cy, cx:].copy()
bottom20 = img[int(h * 0.8):, :].copy()
left_third = img[:, :w // 3].copy()

# Circular ROI = a mask, not a slice (ellipse: cv2.ellipse(mask, (cx, cy), (ax, ay), 0, 0, 360, 255, -1))
r = min(h, w) // 3
mask = np.zeros((h, w), np.uint8)
cv2.circle(mask, (cx, cy), r, 255, -1)
circle_only = cv2.bitwise_and(img, img, mask=mask)

# Apply an effect ONLY inside the circle
k = 25
blurred = cv2.GaussianBlur(img, (k, k), 0)
blur_inside = np.where(mask[..., None] == 255, blurred, img) # [..., None] broadcasts over channels

show([center_roi, center_pct, top_right, bottom20, circle_only, blur_inside],
     [f"Centre {rw}x{rh}", f"Central {N}%", "Top-right quadrant", "Bottom 20%",
      f"Circle r={r}", f"Blur k={k} inside circle"], cols=3)
cv2.imwrite("output/roi_blur_inside_circle.png", blur_inside)

```

2.3 Live tuning with trackbars

`cv2.imshow` windows open as separate windows from a local VS Code notebook (not in Colab). Press `q` or `Esc` to close and print the final values.

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/smarties.png")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
win = "HSV tuner"
cv2.namedWindow(win)
for name, init, mx in [("H_lo", 20, 179), ("S_lo", 100, 255), ("V_lo", 100, 255),
                      ("H_hi", 35, 179), ("S_hi", 255, 255), ("V_hi", 255, 255)]:
    cv2.createTrackbar(name, win, init, mx, lambda v: None) # OpenCV hue range is 0-179

while True:
    g = lambda n: cv2.getTrackbarPos(n, win)
    lo = np.array([g("H_lo"), g("S_lo"), g("V_lo")])
    hi = np.array([g("H_hi"), g("S_hi"), g("V_hi")])
    mask = cv2.inRange(hsv, lo, hi)
    cv2.imshow(win, np.hstack([img, cv2.bitwise_and(img, img, mask=mask)]))
    if cv2.waitKey(30) & 0xFF in (ord("q"), 27):
        break
cv2.destroyAllWindows()
print("lower", lo, "upper", hi)
result = cv2.bitwise_and(img, img, mask=mask)
show([img, mask, result], ["Original", f"Mask lo={lo.tolist()} hi={hi.tolist()}", "Extracted"])
cv2.imwrite("output/hsv_tuned_mask.png", mask)

```

For Canny, use trackbars `low` and `high` (0 to 500) and the body `cv2.Canny(gray, g("low"), g("high"))`. For an odd kernel size from a trackbar use $k = 2t + 1$.

No popup windows allowed? Use an in-notebook slider (needs `ipywidgets`, installed in 1.1):

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
from ipywidgets import interact
gray = load("data/sample.jpg", cv2.IMREAD_GRAYSCALE)

@interact(low=(0, 255, 5), high=(0, 255, 5))
def tune(low=50, high=150):
    edges = cv2.Canny(gray, low, high)
    show([gray, edges], ["Gray", f"Canny low={low}, high={high}"])
    cv2.imwrite(f"output/canny_{low}_{high}.png", edges)

```

2.4 Contours: area, perimeter, box, hull, centroid

Contours need a **binary** image (white objects on black). The centroid comes from image moments $M_{pq} = \sum_x \sum_y x^p y^q I(x, y)$:

$$\bar{x} = \frac{M_{10}}{M_{00}}, \quad \bar{y} = \frac{M_{01}}{M_{00}}, \quad M_{00} = \text{area}$$

Shape scores used as filters:

$$\text{aspect} = \frac{w}{h}, \quad \text{solidity} = \frac{A}{A_{\text{hull}}}, \quad \text{circularity} = \frac{4\pi A}{P^2}$$

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/sample.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, bw = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU) # objects white
cnts, _ = cv2.findContours(bw, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE) # OpenCV 4: 2 return values
min_area = 200
out = img.copy()
kept = 0
for c in cnts:
    area = cv2.contourArea(c)
    if area < min_area:
        continue # filter out specks
    peri = cv2.arcLength(c, True) # True = closed curve
    x, y, bw_, bh = cv2.boundingRect(c)
    aspect = bw_ / float(bh)
    hull = cv2.convexHull(c)
    solidity = area / cv2.contourArea(hull)
    circularity = 4 * np.pi * area / peri ** 2
    M = cv2.moments(c)
    cx, cy = int(M["m10"] / M["m00"]), int(M["m01"] / M["m00"])
    approx = cv2.approxPolyDP(c, 0.02 * peri, True) # 4 points -> rectangle-like
    cv2.rectangle(out, (x, y), (x + bw_, y + bh), (0, 255, 0), 2)
    cv2.drawContours(out, [hull], -1, (255, 0, 0), 1)
    cv2.circle(out, (cx, cy), 4, (0, 0, 255), -1)
    kept += 1
    print(f"#{kept}: area={area:.0f} peri={peri:.0f} AR={aspect:.2f} solid={solidity:.2f} "
          f"circ={circularity:.2f} centroid=({cx},{cy}) corners={len(approx)}")
show([img, bw, out], ["Original", "Otsu binary (inverted)", f"{kept} objects, area >= {min_area}"])
cv2.imwrite("output/contours_annotated.png", out)
```

- Rotated box: `rect = cv2.minAreaRect(c)`, `box = np.int32(cv2.boxPoints(rect))`, `cv2.drawContours(out, [box], 0, (0, 255, 255), 2)`.
- Enclosing circle: `(x, y), r = cv2.minEnclosingCircle(c)`.
- `RETR_EXTERNAL` = outer outlines only; `RETR_TREE` = include holes.
- **Boxes around a colour:** build `mask = cv2.inRange(hsv, lo, hi)`, clean it with `cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))`, then run the loop above on `mask` instead of `bw`.

2.5 Batch: process every image in a folder

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
from pathlib import Path
exts = {".png", ".jpg", ".jpeg", ".bmp", ".tif"}
lo, hi = 50, 150
previews, titles = [], []
for p in sorted(Path("data").iterdir()):
    if p.suffix.lower() not in exts:
        continue
    try:
        im = load(p)
    except FileNotFoundError:
        print("skip unreadable:", p.name) # corrupted files
        continue
```



```

edges = cv2.Canny(cv2.cvtColor(im, cv2.COLOR_BGR2GRAY), lo, hi)
cv2.imwrite(f"output/{p.stem}_edges.png", edges)
previews.append(edges)
titles.append(f"{p.name}: Canny {lo}/{hi}")
if previews:
    show(previews[:6], titles[:6], cols=3)

```

2.6 Video: robust read loop, FPS overlay, writer

```

import cv2, numpy as np, matplotlib.pyplot as plt, time # label() from Section 1.4
cap = cv2.VideoCapture("data/echo.mp4") # 0 = webcam
if not cap.isOpened():
    raise FileNotFoundError("Error: could not open video data/echo.mp4")
fps_in = cap.get(cv2.CAP_PROP_FPS) or 30 # some files report 0
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
writer = cv2.VideoWriter("output/processed.mp4", cv2.VideoWriter_fourcc(*"mp4v"), fps_in, (w * 2, h))
prev, misses, last = time.time(), 0, None
lo, hi = 50, 150
while True:
    ok, frame = cap.read()
    if not ok: # end of file OR a corrupted frame
        misses += 1
        if misses > 5:
            break
        continue
    misses = 0
    proc = cv2.cvtColor(cv2.Canny(frame, lo, hi), cv2.COLOR_GRAY2BGR) # writer needs 3 channels
    combo = np.hstack([frame, proc]) # equal height and channels required
    now = time.time()
    fps = 1 / max(now - prev, 1e-6)
    prev = now
    label(combo, f"FPS {fps:.1f} | Canny {lo}/{hi}")
    writer.write(combo) # size MUST equal the writer's (w*2, h)
    cv2.imshow("Raw | Processed", combo)
    last = combo
    if cv2.waitKey(int(1000 / fps_in)) & 0xFF == ord("q"):
        break
cap.release()
writer.release()
cv2.destroyAllWindows()
if last is not None:
    show([last], [f"Last frame, Canny {lo}/{hi}"])
    cv2.imwrite("output/last_frame.png", last)

```

A 0 KB output video means the frame size or channel count did not match the writer. Fallback codec:

`cv2.VideoWriter_fourcc(*"XVID")` with an `.avi` file.

3. Lab 01: Fundamentals & basic operations

Lab 1 tests whether you can load, convert, crop, annotate, threshold, rotate, blend and equalize an image, plus plain Python classes and dictionaries.

3.1 Theory for MCQs

A digital image is a 2-D function $f(x, y)$ whose coordinates and amplitude are finite and discrete. It is stored as an $M \times N$ array (M rows = height, N columns = width) with the origin $(0, 0)$ at the **top-left**.

$$f(x, y) = \begin{bmatrix} f(0, 0) & f(0, 1) & \cdots & f(0, N-1) \\ f(1, 0) & f(1, 1) & \cdots & f(1, N-1) \\ \vdots & \vdots & \ddots & \vdots \\ f(M-1, 0) & f(M-1, 1) & \cdots & f(M-1, N-1) \end{bmatrix}$$

Term	One-line meaning
Pixel	Smallest unit (picture element); holds colour or intensity.
Resolution	Width × height in pixels, e.g. 1920 × 1080.
Grayscale	One channel, 0 (black) to 255 (white) for 8-bit.
RGB / BGR	Three channels; OpenCV stores BGR , Matplotlib expects RGB.
Histogram	Count of pixels at each intensity level.
Feature / descriptor	A distinctive part of an image / a compact vector describing it.
Segmentation	Splitting an image into meaningful regions.
Image registration	Aligning images of the same scene to one coordinate system.
Optical flow	Motion of pixels between consecutive video frames.
Homography	Transform relating two images of the same planar surface.
SIFT / SURF / HOG	Scale-invariant keypoints / faster SIFT-like / gradient-orientation descriptor.

Fundamental steps of DIP (11): image acquisition, enhancement, restoration, colour image processing, wavelets and multi-resolution processing, compression, morphological processing, segmentation, representation and description, object recognition, all coordinated by the knowledge base.

- Enhancement is subjective (make it look better). Restoration is objective (undo a known degradation such as blur or noise).
- Segmentation is called the hardest step. Representation and description follow it; recognition assigns labels from descriptors.
- Five purposes of DIP: visualization, sharpening and restoration, retrieval, measurement of pattern, recognition.

Libraries (MCQ bait): OpenCV = general CV, C++/Python, high abstraction. scikit-image = image processing, medium. Pillow = basic image I/O, low. TorchVision and Detectron2 = PyTorch (Detectron2 = object detection). Keras = high-level API inside TensorFlow. OpenVINO = Intel inference optimisation. Caffe = C++ CNN framework.

3.2 Core operations (manual 11.1 to 11.12)

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/sukuna.jpeg") # BGR, uint8, shape (H, W, 3)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
h, w = img.shape[:2]

fx = 0.5
half = cv2.resize(img, None, fx=fx, fy=fx) # scale factors keep aspect ratio
fixed = cv2.resize(img, (640, 480)) # exact (width, height)
k = 31
blur = cv2.GaussianBlur(img, (k, k), 0) # odd kernel; sigma 0 = derived from size
left = img[:, :w // 2].copy() # crop = slicing [y, x]
texted = img.copy()
cv2.putText(texted, "Domain Expansion", (50, 80), cv2.FONT_HERSHEY_SIMPLEX, 1.5, (0, 0, 255), 2)
T = 100
ret, th = cv2.threshold(gray, T, 255, cv2.THRESH_BINARY) # > T -> 255 else 0
angle, scale = 45, 1.0
M = cv2.getRotationMatrix2D((w / 2, h / 2), angle, scale) # centre, angle (+ = CCW), scale
rot = cv2.warpAffine(img, M, (w, h)) # corners cut off, filled black
eq = cv2.equalizeHist(gray) # 8-bit single channel only

show([img, gray, blur, left, textured, th, rot, eq],
     ["Original", "Grayscale", f"Gaussian k={k}", "Left half crop", "putText",
      f"Binary T={T}", f"Rotate {angle} deg, scale={scale}", "equalizeHist"], cols=4)
print("Original (w, h):", (w, h), "| resized:", half.shape[1:-1], "| fixed:", fixed.shape[1:-1])
cv2.imwrite("output/lab1_rotated.png", rot)
cv2.imwrite("output/lab1_equalized.png", eq)
```

Blending (11.11): both images must have the same size first. `cv2.add` saturates and brightens; `cv2.addWeighted` mixes with weights α, β .

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img1 = load("data/sukuna.jpeg")
img2 = load("data/gojo.jpeg")
img1 = cv2.resize(img1, (img2.shape[1], img2.shape[0])) # match img2's (width, height)
a, b = 0.6, 0.4
added = cv2.add(img1, img2) # saturating sum: brighter, clips at 255
mixed = cv2.addWeighted(img1, a, img2, b, 0)
show([img1, img2, added, mixed], ["img1", "img2", "cv2.add (saturating)", f"addWeighted a={a}, b={b}"])
cv2.imwrite("output/lab1_add.png", added)
cv2.imwrite("output/lab1_addweighted.png", mixed)
```

Pandas statistics (11.8): reshape $(H, W, 3) \rightarrow (H \cdot W, 3)$, one row per pixel.

```
import cv2, numpy as np, matplotlib.pyplot as plt, pandas as pd # load(), show() from Section 1.4
img = load("data/sukuna.jpeg")
df = pd.DataFrame(img.reshape(-1, 3), columns=["B", "G", "R"]) # BGR order!
print(df.describe()) # count, mean, std, min, 25%, 50%, 75%, max
df.describe().to_csv("output/lab1_stats.csv")
show([img], [f"Mean B={df.B.mean():.1f}, G={df.G.mean():.1f}, R={df.R.mean():.1f}"])
cv2.imwrite("output/lab1_stats_image.png", img)
```

3.3 Task 1: **GroceryManager** (OOP + error handling)

Pure Python, no image: the result is printed, not saved.

```
class GroceryManager:
    def __init__(self):
        self.items = {} # name -> {"quantity": int, "price": float}

    def add_item(self, item, quantity, price):
        if quantity <= 0 or price < 0:
            raise ValueError("Quantity must be > 0 and price >= 0")
        if item in self.items: # adding again increases the quantity
            self.items[item]["quantity"] += quantity
            self.items[item]["price"] = price
        else:
            self.items[item] = {"quantity": quantity, "price": price}
        print(f"Added {quantity} x {item} @ {price}")

    def remove_item(self, item):
        try:
            del self.items[item]
            print(f"Removed {item}")
        except KeyError:
            print(f"Error: '{item}' is not in the list") # required error handling

    def view_list(self):
        if not self.items:
            print("List is empty")
            return
        print(f"{'Item':<12}{'Qty':>5}{'Price':>10}{'Subtotal':>10}")
        for name, d in self.items.items():
            print(f"{'name':<12}{d['quantity']:>5}{d['price']:>10.2f}{d['quantity'] * d['price']:>10.2f}")

    def calculate_total(self):
        return sum(d["quantity"] * d["price"] for d in self.items.values())

gm = GroceryManager()
gm.add_item("Milk", 2, 250)
gm.add_item("Eggs", 12, 30)
gm.add_item("Bread", 1, 180)
gm.remove_item("Butter") # triggers the error message
gm.view_list()
print("Total:", gm.calculate_total())
```

3.4 Task 2: nested student records

Pure Python, no image: the result is printed, not saved.

```

students = {
    "23K-0001": {"Name": "Ali", "Major": "AI", "Grades": [85, 90, 78]},
    "23K-0002": {"Name": "Sara", "Major": "CS", "Grades": [92, 88, 95]},
    "23K-0003": {"Name": "Hamza", "Major": "AI", "Grades": [70, 75, 80]},
}

def average(grades):
    return sum(grades) / len(grades) if grades else 0 # guard against empty lists

def top_student(records):
    best_id = max(records, key=lambda sid: average(records[sid]["Grades"]))
    return records[best_id]["Name"]

def search_by_major(records, major):
    found = [r["Name"] for r in records.values() if r["Major"].lower() == major.lower()]
    print(f"{major}: ", " ".join(found) if found else "no students")

print("Highest average:", top_student(students)) # Sara
search_by_major(students, "AI") # Ali, Hamza

```

3.5 Task 3: safe loading + formatted title

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image_path = "data/sukuna.jpeg"
try:
    image = load(image_path)
except FileNotFoundError as e:
    print(e) # prints "Error: could not load image at ..." instead of crashing
else:
    title = "King of Curses - Ryomen Sukuna"
    plt.figure(figsize=(8, 6))
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB)) # BGR -> RGB before plt.imshow
    plt.title(title, fontsize=16, color="darkred", pad=15)
    plt.axis("off")
    plt.show()
    cv2.imwrite("output/task3_loaded.png", image)

```

3.6 Task 4: 800×800 canvas, 5 concentric circles, bounding box, centre

```

import cv2, numpy as np, matplotlib.pyplot as plt # show() from Section 1.4
size = 800
canvas = np.zeros((size, size, 3), dtype=np.uint8) # (height, width, channels), black
cx, cy = canvas.shape[1] // 2, canvas.shape[0] // 2 # (400, 400)
colors = [(0, 0, 255), (255, 255, 255)] # alternate red / white (BGR)
radii = [300, 240, 180, 120, 60] # draw the LARGEST first so smaller ones sit on top
for i, r in enumerate(radii):
    cv2.circle(canvas, (cx, cy), r, colors[i % 2], thickness=-1) # -1 = filled
R = radii[0]
cv2.rectangle(canvas, (cx - R, cy - R), (cx + R, cy + R), (0, 255, 0), 3) # box around outer circle
cv2.circle(canvas, (cx, cy), 5, (255, 0, 0), -1) # mark the centre
print("Canvas centre (x, y):", (cx, cy))
show([canvas], [f"size{x}x{size} target, radii={radii}, centre=({cx},{cy})"])
cv2.imwrite("output/task4_target.png", canvas)

```

Variations: rings only → `thickness=3`. Any number of rings → `radii = np.linspace(300, 60, n).astype(int)`.

Rainbow colours → sample `cv2.applyColorMap(np.uint8([[i * 255 // n]]), cv2.COLORMAP_HSV)[0, 0]` for each ring *i*.

3.7 Task 5: heavy blur + 300×300 centre ROI from both images

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/highres.jpg")
k, roi = 25, 300
blurred = cv2.GaussianBlur(image, (k, k), 0)
h, w = image.shape[:2]
x1, y1 = w // 2 - roi // 2, h // 2 - roi // 2 # top-left of the centre box

```

```

roi_orig = image[y1:y1 + roi, x1:x1 + roi]
roi_blur = blurred[y1:y1 + roi, x1:x1 + roi]
show([roi_orig, roi_blur], [f"Original centre ROI {roi}x{roi}", f"Blurred ROI, Gaussian k={k}"])
cv2.imwrite("output/task5_roi_original.png", roi_orig)
cv2.imwrite("output/task5_roi_blurred.png", roi_blur)

```

Curveball "blur only the centre, keep the rest sharp": `out = image.copy()`, then `out[y1:y1+roi, x1:x1+roi] = cv2.GaussianBlur(out[y1:y1+roi, x1:x1+roi], (k, k), 0)`.

3.8 Task 6: semi-transparent caption bar on the bottom 20%

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/sukuna.jpeg")
h, w = image.shape[:2]
alpha, frac = 0.4, 0.2 # 0 = invisible bar, 1 = solid bar
overlay = image.copy()
cv2.rectangle(overlay, (0, int(h * (1 - frac))), (w, h), (255, 0, 0), -1) # solid blue bar (BGR)
blended = cv2.addWeighted(overlay, alpha, image, 1 - alpha, 0)
text = "King of Curses"
scale = w / 600 # text grows with image width
(tw, tht), _ = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX, scale, 2)
org = ((w - tw) // 2, int(h * (1 - frac / 2)) + tht // 2) # centred inside the bar
cv2.putText(blended, text, org, cv2.FONT_HERSHEY_SIMPLEX, scale, (255, 255, 255), 2, cv2.LINE_AA)
show([image, blended], ["Original", f"Bottom {int(frac * 100)}% bar, alpha={alpha}"])
cv2.imwrite("output/task6_caption.png", blended)

```

The bar blend per pixel is $out = \alpha \cdot overlay + (1 - \alpha) \cdot image$.

3.9 Task 7: global vs adaptive threshold, then rotate 45° at scale 0.8

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
gray = load("data/document.jpg", cv2.IMREAD_GRAYSCALE)
T, block, C = 127, 11, 2
_, glob = cv2.threshold(gray, T, 255, cv2.THRESH_BINARY)
adapt = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, block, C)
h, w = adapt.shape
angle, scale = 45, 0.8
M = cv2.getRotationMatrix2D((w / 2, h / 2), angle, scale) # 0.8 shrink keeps corners inside
rotated = cv2.warpAffine(adapt, M, (w, h), borderValue=255) # white fill
show([gray, glob, adapt, rotated],
     ["Gray", f"Global T={T}", f"Adaptive Gaussian block={block}, C={C}", f"Rotated {angle} deg, scale={scale}"])
cv2.imwrite("output/task7_adaptive_rotated.png", rotated)

```

Why 0.8? A square rotated 45° needs a canvas $\sqrt{2} \approx 1.41$ times wider, so shrinking keeps most corners visible. To keep the whole image at scale 1, enlarge the canvas instead (Section 5.6).

3.10 Task 8: bitwise masking (AND, NOT, OR, XOR)

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
a = load("data/sukuna.jpeg")
b = load("data/gojo.jpeg")
size = 500
a = cv2.resize(a, (size, size))
b = cv2.resize(b, (size, size))
mask = np.zeros((size, size), np.uint8) # 1-channel, black
pts = np.array([[250, 60], [440, 200], [370, 440], [130, 440], [60, 200]], np.int32) # pentagon
cv2.fillPoly(mask, [pts], 255)
fg = cv2.bitwise_and(a, a, mask=mask) # shape cut from image A
inv = cv2.bitwise_not(mask) # 255 - mask
bg = cv2.bitwise_and(b, b, mask=inv) # background from image B
combined = cv2.bitwise_or(fg, bg)
r = 150
circle = cv2.circle(np.zeros_like(mask), (size // 2, size // 2), r, 255, -1)
xor = cv2.bitwise_xor(mask, circle) # where pentagon and circle differ
show([a, b, mask, fg, bg, combined, xor],

```

```
[ "A", "B", "Pentagon mask", "A AND mask", "B AND NOT mask", "OR (combined)", f"Pentagon XOR circle r={r}" ],
cols=4)
cv2.imwrite("output/task8_combined.png", combined)
cv2.imwrite("output/task8_xor.png", xor)
```

Op	Rule per bit	Use
AND	1 only if both are 1	Keep pixels inside a mask
OR	1 if either is 1	Merge two non-overlapping layers
NOT	flips 0 ↔ 255	Invert a mask
XOR	1 if exactly one is 1	Show where two masks differ

The `mask` argument must be uint8, single channel, and the same height and width as the image.

3.11 Task 9: per-channel statistics with Pandas

```
import cv2, numpy as np, matplotlib.pyplot as plt, pandas as pd # load(), show() from Section 1.4
img = load("data/sukuna.jpeg")
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
R, G, B = rgb[:, :, 0].flatten(), rgb[:, :, 1].flatten(), rgb[:, :, 2].flatten() # 2-D -> 1-D
df = pd.DataFrame({"Red": R, "Green": G, "Blue": B})
summary = df.describe()
print(summary.loc[["mean", "std", "min", "max"]])
print("Brightest channel on average:", df.mean().idxmax())
channels = [img[:, :, 2], img[:, :, 1], img[:, :, 0]] # R, G, B as grayscale images
show([img] + channels, ["Original"] + [f"{n}: mean={df[n].mean():.1f}, std={df[n].std():.1f}" for n in
df.columns])
summary.to_csv("output/task9_channel_stats.csv")
cv2.imwrite("output/task9_red_channel.png", channels[0])
```

Each column's mean and standard deviation are $\mu = \frac{1}{n} \sum_i x_i$ and $\sigma = \sqrt{\frac{1}{n-1} \sum_i (x_i - \mu)^2}$ (Pandas uses $n - 1$).

4. Lab 02: Photometric (point) transforms & medical imaging

A point transform changes each pixel's value using only that pixel's value, $s = T(r)$, never its neighbours. Sir Talha named Log, Gamma, Piecewise and Histogram as the four to study.

4.1 The big picture

- r = input intensity, s = output intensity, both in $[0, L - 1]$ with $L = 256$ for 8-bit images.
- **Geometric** transforms move pixels (where). **Photometric** transforms change values (what). Point transforms are photometric.
- A point transform is a lookup table of 256 entries, so `cv2.LUT(img, table)` applies any of them fast.

Transform	Formula	Effect	Typical use
Negative	$s = L - 1 - r$	Inverts	Mammograms, white detail in dark areas
Log	$s = c \log(1 + r)$	Strongly brightens darks	Huge dynamic range (Fourier spectra), dark X-rays
Inverse log	$s = c(e^r - 1)$	Brightens brights	Opposite of log
Gamma, $\gamma < 1$	$s = c r^\gamma$	Brightens darks and midtones	Underexposed images, soften bone glare
Gamma, $\gamma > 1$	$s = c r^\gamma$	Darkens	Washed-out images
Piecewise linear	Lines through $(r_1, s_1), (r_2, s_2)$	You choose	Contrast stretching, thresholding, slicing
Histogram equalization	$s_k = (L - 1) \text{CDF}(r_k)$	Spreads clumped levels	Low-contrast images, automatic

4.2 Log transformation

$$s = c \cdot \log(1 + r), \quad c = \frac{255}{\log(1 + r_{\max})}$$

The +1 avoids $\log(0)$. The constant c rescales so the brightest input maps to 255. Log stretches dark values (roughly 0–50 spreads over 0–180) and compresses bright ones.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
gray = load("data/xray.png", cv2.IMREAD_GRAYSCALE)
f = gray.astype(np.float32) # float avoids uint8 overflow
c = 255 / np.log(1 + f.max())
log_img = np.clip(c * np.log1p(f), 0, 255).astype(np.uint8) # log1p(x) = log(1 + x)
show([gray, log_img], ["Original", f"Log, c={c:.1f}"])
cv2.imwrite("output/log_transform.png", log_img)
```

4.3 Power-law (gamma) transformation

$$s = c \cdot r^\gamma, \quad r \in [0, 1] \text{ (normalise first)}, \quad c = 1$$

- $\gamma < 1$ (e.g. 0.4, 0.5): the curve bows upward, so darks brighten and harsh highlights dominate less. Lab 2 asks for this.
- $\gamma = 1$: identity. $\gamma > 1$ (e.g. 2.2): darkens and restores washed-out images.
- Normalising to $[0, 1]$ matters: $255^{0.5} \approx 15.97$, which would make everything nearly black.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show(), gamma_correct() from Section 1.4
gray = load("data/xray.png", cv2.IMREAD_GRAYSCALE)
gammas = [0.4, 0.5, 1.0, 2.2]
outs = [gamma_correct(gray, g) for g in gammas]
show(outs, [f"Gamma={g}" for g in gammas])
for g, o in zip(gammas, outs):
    cv2.imwrite(f"output/gamma_{g}.png", o)
```

4.4 Piecewise-linear transformations

Contrast stretching with control points (r_1, s_1) and (r_2, s_2) :

$$s = \begin{cases} \frac{s_1}{r_1} r & 0 \leq r < r_1 \\ \frac{s_2 - s_1}{r_2 - r_1} (r - r_1) + s_1 & r_1 \leq r \leq r_2 \\ \frac{255 - s_2}{255 - r_2} (r - r_2) + s_2 & r_2 < r \leq 255 \end{cases}$$

Special cases: $r_1 = s_1$ and $r_2 = s_2$ gives identity; $r_1 = r_2$ with $s_1 = 0$, $s_2 = 255$ gives a threshold; $r_1 = r_{\min}$, $r_2 = r_{\max}$, $s_1 = 0$, $s_2 = 255$ gives min-max stretching:

$$s = \frac{r - r_{\min}}{r_{\max} - r_{\min}} \times 255$$

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
gray = load("data/xray.png", cv2.IMREAD_GRAYSCALE)

def piecewise(img, r1, s1, r2, s2):
    r = np.arange(256, dtype=np.float32)
    table = np.interp(r, [0, r1, r2, 255], [0, s1, s2, 255]) # straight lines between control points
    return cv2.LUT(img, np.clip(table, 0, 255).astype(np.uint8)), table

r1, s1, r2, s2 = 70, 20, 180, 235
stretched, table = piecewise(gray, r1, s1, r2, s2)
minmax = cv2.normalize(gray, None, 0, 255, cv2.NORM_MINMAX) # min-max stretch
A, B = 100, 160
slice_bin = np.where((gray >= A) & (gray <= B), 255, 0).astype(np.uint8) # band white, rest black
```

```

slice_keep = np.where((gray >= A) & (gray <= B), 255, gray).astype(np.uint8) # band white, rest kept
negative = 255 - gray
k = 7
bitplane = (((gray >> k) & 1) * 255).astype(np.uint8) # bit-plane k (0 = LSB, 7 = MSB)

plt.figure(figsize=(4, 4))
plt.plot(np.arange(256), table)
plt.xlabel("input r")
plt.ylabel("output s")
plt.title(f"Transfer curve ({r1},{s1}) ({r2},{s2})")
plt.show()
show([gray, stretched, minmax, slice_bin, slice_keep, negative, bitplane],
     ["Original", f"Stretch ({r1},{s1})-({r2},{s2})", "Min-max stretch", f"Slice [{A},{B}] binary",
      f"Slice [{A},{B}] keep rest", "Negative", f"Bit-plane {k}"], cols=4)
cv2.imwrite("output/piecewise_stretch.png", stretched)
cv2.imwrite("output/intensity_slice.png", slice_keep)

```

4.5 Histogram processing

The normalised histogram is $p_r(r_k) = \frac{n_k}{MN}$. Equalization maps each level through the cumulative distribution:

$$s_k = (L - 1) \sum_{j=0}^k p_r(r_j) = \frac{L - 1}{MN} \sum_{j=0}^k n_j$$

Intuition: if most pixels sit in 40–80, the CDF rises steeply there, so those levels get spread across 0–255. The result is a roughly flat histogram and higher global contrast. Downside: it also amplifies noise and can over-brighten backgrounds; CLAHE fixes that by equalizing small tiles with a clip limit.

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
gray = load("data/xray.png", cv2.IMREAD_GRAYSCALE)
eq = cv2.equalizeHist(gray) # 8-bit single channel only
clip, tiles = 2.0, (8, 8)
clahe = cv2.createCLAHE(clipLimit=clip, tileGridSize=tiles).apply(gray)

# Manual equalization (if told "do not use equalizeHist")
hist = np.bincount(gray.ravel(), minlength=256)
cdf = hist.cumsum()
cdf_min = cdf[cdf > 0][0]
lut = np.round((cdf - cdf_min) / (cdf[-1] - cdf_min) * 255).clip(0, 255).astype(np.uint8)
eq_manual = lut[gray] # fancy indexing applies the LUT

titles = ["Raw", "equalizeHist", f"CLAHE clip={clip}, tiles={tiles}", "Manual CDF"]
fig, ax = plt.subplots(2, 4, figsize=(18, 8))
for i, (im, t) in enumerate(zip([gray, eq, clahe, eq_manual], titles)):
    ax[0, i].imshow(im, cmap="gray") # single channel: cmap gray, no BGR issue
    ax[0, i].set_title(t)
    ax[0, i].axis("off")
    ax[1, i].hist(im.ravel(), bins=256, range=(0, 256), color="k")
    c = np.cumsum(np.bincount(im.ravel(), minlength=256))
    ax[1, i].plot(c / c[-1] * ax[1, i].get_ylim()[1], "r")
    ax[1, i].set_title(f"{t}: histogram + CDF")
plt.tight_layout()
plt.show()
cv2.imwrite("output/hist_equalized.png", eq)
cv2.imwrite("output/hist_clahe.png", clahe)

```

Colour image equalization: never equalize B, G, R separately (colours shift). Equalize the luminance channel only:

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/sample.jpg")
yrcb = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
yrcb[:, :, 0] = cv2.equalizeHist(yrcb[:, :, 0]) # Y = luminance
color_eq = cv2.cvtColor(yrcb, cv2.COLOR_YCrCb2BGR)
show([img, color_eq], ["Original", "Equalized Y channel (YCrCb)"])
cv2.imwrite("output/color_equalized.png", color_eq)

```


`cv2.calcHist([gray], [0], None, [256], [0, 256])` returns the histogram as a 256×1 array.

4.6 Colour mapping and colour balance

`cv2.applyColorMap(gray8, cv2.COLORMAP_JET)` maps low values to blue, mid to green/yellow and high to red. Eyes separate hues far better than shades of grey, so a small intensity step at a fluid boundary becomes a visible colour change.

Gray-world colour balance: assume the scene averages to neutral grey, so scale each channel to the common mean $\bar{K}$:

$$B' = B \cdot \frac{\bar{K}}{\bar{B}}, \quad G' = G \cdot \frac{\bar{K}}{\bar{G}}, \quad R' = R \cdot \frac{\bar{K}}{\bar{R}}, \quad \bar{K} = \frac{\bar{B} + \bar{G} + \bar{R}}{3}$$

Percentile balance: stretch each channel so its p -th and $(100 - p)$ -th percentiles hit 0 and 255.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show(), gray_world() from Section 1.4
gray = load("data/xray.png", cv2.IMREAD_GRAYSCALE)
heat = cv2.applyColorMap(cv2.equalizeHist(gray), cv2.COLORMAP_JET) # output is BGR

def percentile_balance(img, pct=1):
    out = np.zeros_like(img)
    for ch in range(3):
        lo, hi = np.percentile(img[:, :, ch], (pct, 100 - pct))
        out[:, :, ch] = np.clip((img[:, :, ch] - lo) * 255.0 / max(hi - lo, 1), 0, 255)
    return out

pct = 1
gw = gray_world(heat)
pb = percentile_balance(heat, pct)
show([gray, heat, gw, pb], ["Gray", "JET colormap", "Gray-world balance", f"Percentile balance p={pct}"])
cv2.imwrite("output/jet_grayworld.png", gw)
```

4.7 Task 1: chest X-ray enhancement pipeline

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show(), gamma_correct(), gray_world() from Section 1.4
raw = load("data/sample_xray.png", cv2.IMREAD_GRAYSCALE) # relative path
raw = cv2.resize(raw, (512, 512)) # Kaggle images vary in size

T, g = 200, 0.5
eq = cv2.equalizeHist(raw) # 1 contrast enhancement
heat = cv2.applyColorMap(eq, cv2.COLORMAP_JET) # 2 false colour
balanced = gray_world(heat) # 3 colour balance
_, dense = cv2.threshold(eq, T, 255, cv2.THRESH_BINARY) # 4 keep only densest tissue
f = raw.astype(np.float32)
log_img = np.clip(255 / np.log1p(f.max()) * np.log1p(f), 0, 255).astype(np.uint8) # 5 reveal faint ribs
gam = gamma_correct(raw, g) # 6 gamma < 1 softens bone glare

steps = [raw, eq, heat, balanced, dense, log_img, gam]
names = ["raw", "equalized", "jet", "grayworld", f"threshold_{T}", "log", f"gamma_{g}"]
show(steps, ["Raw", "Equalized", "JET colormap", "Gray-world", f"Threshold T={T}", "Log", f"Gamma={g}"], cols=4)
for i, (im, n) in enumerate(zip(steps, names)):
    cv2.imwrite(f"output/xray_{i}_{n}.png", im)
```

Analysis to write: equalization redistributes clumped dark values so lung fields and fluid become visible. JET turns subtle grey steps into hue changes. $T = 200$ isolates bone and dense fluid (use Otsu if unsure). Log expands the dark background to reveal faint rib edges. $\gamma = 0.5$ lifts midtones so soft lung tissue is not drowned by bright bone.

4.8 Task 2: CT + MRI weighted fusion

$$F = \alpha \cdot \text{CT} + \beta \cdot \text{MRI}, \quad \alpha + \beta = 1, \quad \alpha > \beta$$

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show(), gamma_correct() from Section 1.4
ct = load("data/ct_01.png", cv2.IMREAD_GRAYSCALE)
mri = load("data/mri_01.png", cv2.IMREAD_GRAYSCALE)
mri = cv2.resize(mri, (ct.shape[1], ct.shape[0])) # MUST match before blending

ct_eq, mri_eq = cv2.equalizeHist(ct), cv2.equalizeHist(mri) # independent equalization
ct_col = cv2.cvtColor(ct_eq, cv2.COLOR_GRAY2BGR) # CT kept grey (sharp edges)
mri_col = cv2.applyColorMap(mri_eq, cv2.COLORMAP_JET) # MRI coloured (soft tissue)

alpha, beta = 0.7, 0.3 # CT heavier: sharp anatomy
fused = cv2.addWeighted(ct_col, alpha, mri_col, beta, 0)
f = fused.astype(np.float32)
fused_log = np.clip(255 / np.log1p(f.max()) * np.log1p(f), 0, 255).astype(np.uint8)
g = 1.2
fused_final = gamma_correct(fused_log, g) # > 1 pulls back log's brightening

for a in (0.3, 0.5, 0.7): # numeric validation of the weights
    m = cv2.addWeighted(ct_col, a, mri_col, 1 - a, 0)
    sharp = cv2.Laplacian(cv2.cvtColor(m, cv2.COLOR_BGR2GRAY), cv2.CV_64F).var()
    print(f"alpha={a}: mean={m.mean():.1f}, edge sharpness (Laplacian var)={sharp:.0f}")
show([ct_eq, mri_col, fused, fused_final],
     ["CT equalized", "MRI JET", f"Fused alpha={alpha}, beta={beta}", f"Log + gamma={g}"])
cv2.imwrite("output/fusion_final.png", fused_final)

```

- $\alpha + \beta = 1$ keeps brightness in range; $\alpha + \beta > 1$ risks clipping to white.
- "Mathematically validate" means reporting numbers: means, standard deviations, or edge strength (Laplacian variance: CT high, MRI low, fused in between).
- If the slices were not aligned, fusion would show doubled edges (ghosting). The task assumes perfectly aligned slices.

4.9 Task 3: real-time echocardiogram pipeline

```

import cv2, numpy as np, matplotlib.pyplot as plt # show(), gray_world() from Section 1.4
cap = cv2.VideoCapture("data/echo_sample.avi") # EchoNet clips are .avi; .mp4 works the same
if not cap.isOpened():
    raise FileNotFoundError("Error: could not open video data/echo_sample.avi")
g = 1.5 # > 1 suppresses bright backscatter
lut_gamma = np.array([(i / 255) ** g * 255 for i in range(256)], np.uint8)
lut_log = np.array([255 / np.log(256) * np.log1p(i) for i in range(256)], np.uint8)
frame_no, panel = 0, None
while True:
    ok, frame = cap.read()
    if not ok:
        break # end of video
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    eq = cv2.equalizeHist(gray) # fight murky contrast
    heat = cv2.applyColorMap(eq, cv2.COLORMAP_JET) # highlight flow intensities
    bal = gray_world(heat) # colour balance
    logd = cv2.LUT(bal, lut_log) # reveal dark chambers
    out = cv2.LUT(logd, lut_gamma) # suppress white backscatter
    panel = np.hstack([frame, out]) # raw | enhanced (same size, 3 channels)
    panel = cv2.resize(panel, None, fx=3, fy=3) # EchoNet frames are 112x112
    if frame_no == 50:
        cv2.imwrite("output/echo_pipeline_frame50.png", panel)
        frame_no += 1
    cv2.imshow("Raw | Enhanced", panel)
    if cv2.waitKey(30) & 0xFF == ord("q"): # about 33 fps; q to quit
        break
cap.release()
cv2.destroyAllWindows()
if panel is not None:
    show([panel], [f"Last frame: equalize, JET, gray-world, log, gamma={g}"])
    cv2.imwrite("output/echo_pipeline_last.png", panel)

```

- Gamma here is **above** 1 to *suppress* bright backscatter. In the X-ray task it is below 1 to *lift* midtones.
- Without popup windows, save the stream with `cv2.VideoWriter` (Section 2.6).

- README for this task: activate the environment, `pip install -r requirements.txt`, open the notebook, run all, press `q` to stop.

5. Lab 03: Geometric transformations

Geometric transforms move pixels, $(x, y) \rightarrow (x', y')$, without changing their values. OpenCV applies affine maps with a 2×3 matrix (`warpAffine`) and projective maps with a 3×3 matrix (`warpPerspective`).

5.1 Coordinates: the trap that costs marks

- The origin $(0, 0)$ is the **top-left** corner; x grows right, y grows **down**. A 640×480 image has corners $(0, 0)$, $(639, 0)$, $(0, 479)$, $(639, 479)$.
- NumPy indexing is `image[y, x]` (row, column). Geometric points and every OpenCV function take (x, y) .
- Because y points down, a positive angle in `getRotationMatrix2D` turns the picture **counter-clockwise on screen**.
- 2D transforms use 2×3 or 3×3 matrices. 3D transforms (x, y, z) use 4×4 homogeneous matrices. Mapping a 3D world point (X, Y, Z) to an image point (x, y) is camera projection, a separate topic.

5.2 Reading a 2×2 matrix: columns are the new axes

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \quad \mathbf{A} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} a \\ c \end{bmatrix} \text{ (new } x\text{-axis)}, \quad \mathbf{A} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} b \\ d \end{bmatrix} \text{ (new } y\text{-axis)}$$

Read the matrix by **columns**: the first column is where the unit arrow $\hat{x} = (1, 0)$ lands, the second is where $\hat{y} = (0, 1)$ lands. Every other point follows along.

Entry	Role	Visual effect
a	x -component of the new x -axis	Horizontal scaling; $a < 0$ flips horizontally
c	y -component of the new x -axis	Vertical shear (tilts vertical grid lines)
b	x -component of the new y -axis	Horizontal shear (tilts horizontal grid lines)
d	y -component of the new y -axis	Vertical scaling; $d < 0$ flips vertically

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow x' = ax + by, \quad y' = cx + dy$$

5.3 The four linear building blocks

$$\underbrace{\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}}_{\text{scale}} \quad \underbrace{\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}}_{\text{rotation}} \quad \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}}_{\text{reflect across } x\text{-axis}} \quad \underbrace{\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}}_{\text{horizontal shear}}$$

- **Scaling**: values on the diagonal only, so the axes stretch independently with no mixing.
- **Rotation**: all four entries change together through $\sin \theta$ and $\cos \theta$, keeping the axes perpendicular and unit length. You cannot rotate by changing one entry.
- **Reflection**: a negative diagonal entry flips that axis; $\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$ mirrors left-right.
- **Shear**: $x' = x + ky$, $y' = y$. Rows slide sideways more the further down they are.

5.4 Why linear maps cannot translate, and how homogeneous coordinates fix it

$$\mathbf{A} \mathbf{0} = \mathbf{0} \quad \text{for every } 2 \times 2 \text{ matrix } \mathbf{A}$$

The origin is pinned: multiplying $(0, 0)$ by any 2×2 matrix gives $(0, 0)$. A linear map can spin, stretch and shear the grid around that pin, but cannot slide it. Moving $(10, 20)$ to $(110, 70)$ is impossible with a 2×2 matrix alone.

Affine = linear + translation:

$$p' = \mathbf{A}p + \mathbf{b} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

The $+\mathbf{b}$ is a separate addition, which breaks the pure chain of matrix multiplications that GPUs are optimised for. The trick: append a dummy 1 to every point, $(x, y) \rightarrow (x, y, 1)$, and fold the shift into a third column.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow x' = x + t_x, \quad y' = y + t_y$$

The 1 multiplies t_x and t_y , producing the shift by multiplication alone. Scale, rotation and translation are now all 3×3 , so they **compose** into one master matrix applied once per pixel. Matrices apply right to left, and $\mathbf{TR} \neq \mathbf{RT}$ in general:

$$\mathbf{M}_{\text{total}} = \mathbf{T} \mathbf{R} \mathbf{S} \quad (\text{scale first, then rotate, then translate})$$

`warpAffine` takes only the top two rows (2×3); the bottom row of an affine matrix is always $[0, 0, 1]$.

5.5 Projective transforms (homographies)

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}, \quad y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

The **denominator** $w' = h_{31}x + h_{32}y + h_{33}$ is the whole difference. In affine maps the bottom row is locked to $[0, 0, 1]$, so $w' = 1$ and scaling is uniform. In a homography w' changes with (x, y) : points "further away" get a bigger denominator and squeeze together. That variable division creates depth, converging lines and vanishing points.

$\mathbf{H}$ has 9 entries but is defined only up to scale ($h_{33} = 1$), leaving **8 degrees of freedom**, so it needs **4 point pairs** (each gives 2 equations). An affine map has 6 DOF and needs 3 pairs.

Class	DOF	Matrix	Preserves	OpenCV
Translation	2	$[\mathbf{I} \mid \mathbf{t}]$	Everything except position	build $\mathbf{M}$ by hand
Rigid (Euclidean)	3	$[\mathbf{R} \mid \mathbf{t}]$	Lengths, angles, area	<code>getRotationMatrix2D(c, θ, 1)</code> + shift
Similarity	4	$[s\mathbf{R} \mid \mathbf{t}]$	Angles, shape, ratios	<code>getRotationMatrix2D(c, θ, s)</code>
Affine	6	$[\mathbf{A} \mid \mathbf{t}]$	Parallel lines, collinearity, ratios along a line	<code>getAffineTransform</code> (3 pairs)
Projective	8	$3 \times 3 \mathbf{H}$	Straight lines only (not parallelism)	<code>getPerspectiveTransform</code> (4 pairs)

The hierarchy (manual p.4): 2D geometric transforms split into **Affine** (Linear: rotation, scale, shear; plus Translation) and **Projective**. Rigid $\subset$ Similarity $\subset$ Affine $\subset$ Projective.

Affine guarantees: collinear points stay collinear, parallel lines stay parallel, and the ratio $\overline{AB} : \overline{BC}$ along a line is unchanged. Angles and lengths may change.

5.6 Rotation (with and without cropping)

`getRotationMatrix2D(center, angle, scale)` returns this 2×3 matrix, with $\alpha = s \cos \theta$ and $\beta = s \sin \theta$:

$$\mathbf{M} = \begin{bmatrix} \alpha & \beta & (1 - \alpha)c_x - \beta c_y \\ -\beta & \alpha & \beta c_x + (1 - \alpha)c_y \end{bmatrix}$$

The third column is the translation that keeps the chosen centre (c_x, c_y) fixed. A full-canvas rotation needs a new size:

$$w_{\text{new}} = h |\sin \theta| + w |\cos \theta|, \quad h_{\text{new}} = h |\cos \theta| + w |\sin \theta|$$

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/image.jpg")
h, w = image.shape[:2]
center = (w // 2, h // 2)
```

```

angle = 45
M = cv2.getRotationMatrix2D(center, angle, 1.0) # CCW, no scaling
rotated = cv2.warpAffine(image, M, (w, h)) # dsize = (width, height); corners cropped

cos, sin = abs(M[0, 0]), abs(M[0, 1])
nW, nH = int(h * sin + w * cos), int(h * cos + w * sin)
M_full = M.copy()
M_full[0, 2] += nW / 2 - center[0] # shift so the centre lands mid-canvas
M_full[1, 2] += nH / 2 - center[1]
rotated_full = cv2.warpAffine(image, M_full, (nW, nH), borderValue=(255, 255, 255))
flipped = cv2.flip(image, 1) # 1 horizontal, 0 vertical, -1 both
rot90 = cv2.rotate(image, cv2.ROTATE_90_CLOCKWISE) # exact 90 deg, no black corners
show([image, rotated, rotated_full, flipped, rot90],
     ["Original", f"Rotate {angle} deg (cropped)", f"Rotate {angle} deg, canvas {nW}x{nH}", "Flip code=1",
      "ROTATE_90_CLOCKWISE"], cols=3)
cv2.imwrite("output/rotate_full_canvas.png", rotated_full)

```

5.7 Scaling

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/image.jpg")
h, w = image.shape[:2]
s = 3
scaled = cv2.resize(image, None, fx=s, fy=s, interpolation=cv2.INTER_LINEAR)
S = np.float32([[s, 0, 0], [0, s, 0]]) # the same scaling as a matrix
scaled_m = cv2.warpAffine(image, S, (w * s, h * s))
print(image.shape, "->", scaled.shape) # Matplotlib fits subplots, so print sizes to prove it
show([image, scaled, scaled_m], [f"Original {w}x{h}", f"resize fx=fy={s}", f"warpAffine S={s}"])
cv2.imwrite("output/scaled_3x.png", scaled)

```

5.8 Shear with a widened canvas

With $x' = x + k_x y$, the bottom row ($y = h - 1$) shifts right by about $k_x h$, so the canvas must be $k_x h$ wider: $w_{\text{new}} = w + |k_x| h$.

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/image.jpg")
h, w = image.shape[:2]
kx = 0.5
M = np.float32([[1, kx, 0],
                [0, 1, 0]]) # x' = x + 0.5*y
new_width = int(w + abs(kx) * h)
if kx < 0:
    M[0, 2] = abs(kx) * h # negative shear: shift right so nothing falls off the left
sheared = cv2.warpAffine(image, M, (new_width, h))

ky = 0.3 # vertical shear: y' = y + 0.3*x
My = np.float32([[1, 0, 0], [ky, 1, 0]])
sheared_y = cv2.warpAffine(image, My, (w, int(h + abs(ky) * w)))
show([image, sheared, sheared_y], ["Original", f"X-shear kx={kx}, width {new_width}", f"Y-shear ky={ky}"])
cv2.imwrite("output/shear_x.png", sheared)

```

5.9 Translation

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/image.jpg")
h, w = image.shape[:2]
tx, ty = 100, 50 # right 100, down 50
M = np.float32([[1, 0, tx], [0, 1, ty]])
translated = cv2.warpAffine(image, M, (w, h)) # left and top strips become black
translated_full = cv2.warpAffine(image, M, (w + tx, h + ty)) # bigger canvas keeps everything
show([image, translated, translated_full], ["Original", f"tx={tx}, ty={ty}", f"tx={tx}, ty={ty}, full canvas"])
cv2.imwrite("output/translated.png", translated)

```

Negative t_x moves left; negative t_y moves **up** (y points down).

5.10 Rigid and similarity transforms

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/image.jpg")
h, w = image.shape[:2]
center = (w // 2, h // 2)
angle, tx, ty = 30, 100, 50
M = cv2.getRotationMatrix2D(center, angle, 1.0) # scale MUST be 1.0 for rigid
M[0, 2] += tx # add (+) to the existing centre-keeping offset
M[1, 2] += ty
rigid = cv2.warpAffine(image, M, (w, h))
s = 0.7
Ms = cv2.getRotationMatrix2D(center, angle, s) # scale != 1 -> similarity
similar = cv2.warpAffine(image, Ms, (w, h))
show([image, rigid, similar], ["Original", f"Rigid: {angle} deg, +({tx},{ty})", f"Similarity: {angle} deg, s={s}"])
cv2.imwrite("output/rigid.png", rigid)
```

Use `+=`, not `=`: the third column already holds the offset that keeps the centre fixed.

5.11 General affine from 3 point pairs, composition and inverse

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/image.jpg")
h, w = image.shape[:2]
src = np.float32([[0, 0], [w - 1, 0], [0, h - 1]])
dst = np.float32([[0, h * 0.33], [w * 0.85, h * 0.25], [w * 0.15, h * 0.7]])
A = cv2.getAffineTransform(src, dst) # 2x3, exactly 3 pairs
affine = cv2.warpAffine(image, A, (w, h))

def to3(M2x3):
    return np.vstack([M2x3, [0, 0, 1]]) # 2x3 -> 3x3 homogeneous

s, ang, tx, ty = 0.5, 20, 150, 80
S = np.float32([[s, 0, 0], [0, s, 0], [0, 0, 1]])
R = to3(cv2.getRotationMatrix2D((0, 0), ang, 1.0))
T = np.float32([[1, 0, tx], [0, 1, ty], [0, 0, 1]])
M_total = (T @ R @ S)[:2] # right to left: S first, then R, then T
combo = cv2.warpAffine(image, M_total, (w, h))

restored = cv2.warpAffine(affine, cv2.invertAffineTransform(A), (w, h)) # undo the affine
print("corner (w-1, 0) ->", A @ np.array([w - 1, 0, 1])) # [x', y'] = M [x, y, 1]^T
show([image, affine, combo, restored],
     ["Original", "Affine from 3 pairs", f"T R S: s={s}, {ang} deg, +({tx},{ty})", "Inverse affine"])
cv2.imwrite("output/affine.png", affine)
cv2.imwrite("output/composed_TRS.png", combo)
```

5.12 Perspective (4 points), inverse warp and page flattening

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/image.jpg")
h, w = image.shape[:2]
src_points = np.float32([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]]) # TL, TR, BR, BL
dst_points = np.float32([[100, 50], [w - 150, 0], [w - 50, h - 80], [50, h - 20]])
H = cv2.getPerspectiveTransform(src_points, dst_points) # 3x3, exactly 4 pairs, same order
perspective = cv2.warpPerspective(image, H, (w, h))
back = cv2.warpPerspective(perspective, np.linalg.inv(H), (w, h)) # inverse homography
pt = np.float32([[w - 1, 0]]) # points must be shaped (N, 1, 2)
print("TR corner maps to", cv2.perspectiveTransform(pt, H).ravel()) # does the w' division
show([image, perspective, back], ["Original", "Perspective (4 point pairs)", "Inverse warp H^-1"])
cv2.imwrite("output/perspective.png", perspective)
```

To flatten a photographed page, put its 4 corners (TL, TR, BR, BL) in `src` and a clean rectangle $[(0, 0), (W - 1, 0), (W - 1, H - 1), (0, H - 1)]$ in `dst`. The automatic version is in Section 8.6.

5.13 Lab 3 quick answers

- **Why black regions after warping?** Output pixels whose source falls outside the input get `borderValue` (default 0). Change it with `borderMode=cv2.BORDER_REFLECT` or `borderValue=(255, 255, 255)`.
- **Why does `warpAffine` map "backwards"?** For each output pixel it inverse-maps into the source and interpolates, which avoids holes.
- **Affine vs projective in one line:** affine keeps parallel lines parallel; projective lets them meet at a vanishing point.

6. Lab 04: Feature extraction, filtering & edges

Feature extraction turns a huge pixel array into a short, meaningful vector. Lab 4 covers six histogram descriptors, convolution and its border and stride variants, and edge detection up to Canny.

6.1 Theory in one table

Idea	Meaning
Why extract features	Raw pixels are high-dimensional and noisy; features reduce dimension, keep what matters, cut computation.
Local features	From small regions: keypoints, corners, patches (matching, detection). SIFT, HOG cells, LBP codes.
Global features	Whole-image summaries: colour histogram, texture descriptors, moments.
Technique families	Histogram-based, filter-based (Gabor, Haar wavelets), deep-learning (CNN-learned).
Applications	Object, face, gesture recognition; medical anomalies; autonomous driving.
Challenges	Lighting, scale, orientation, noise.

6.2 HOG (Histogram of Oriented Gradients)

Steps: grayscale → Sobel gradients → magnitude and orientation per pixel → 8×8 cells → 9-bin orientation histogram per cell (0° – 180° , 20° bins, votes weighted by magnitude) → normalise over 2×2 -cell blocks (lighting robustness) → concatenate. Classic use: pedestrian detection.

$$G = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan 2(G_y, G_x)$$

For the classic 64×128 window: 7×15 blocks $\times$ 4 cells $\times$ 9 bins = **3780** features.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
from skimage.feature import hog
from skimage import exposure
image = load("data/image.jpg", cv2.IMREAD_GRAYSCALE)
ori, ppc, cpb = 9, (8, 8), (2, 2)
features, hog_image = hog(image, orientations=ori, pixels_per_cell=ppc,
                          cells_per_block=cpb, block_norm="L2-Hys", visualize=True)
hog_vis = exposure.rescale_intensity(hog_image, in_range=(0, 10), out_range=(0, 255)).astype(np.uint8)
print("HOG feature vector:", features.shape, features[:10].round(3))
show([image, hog_vis], ["Original", f"HOG bins={ori}, cell={ppc}, block={cpb}: {features.size} features"])
cv2.imwrite("output/hog_visualisation.png", hog_vis)
```

Colour input: add `channel_axis=-1` (older scikit-image: `multichannel=True`). Bigger `pixels_per_cell` gives a shorter, coarser vector.

6.3 LBP (Local Binary Pattern)

Compare each pixel's P neighbours on a circle of radius R with the centre g_c : neighbour $\geq$ centre → 1, else 0. Read the bits as a binary number.

$$\text{LBP}_{P,R}(x_c, y_c) = \sum_{p=0}^{P-1} s(g_p - g_c) 2^p, \quad s(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

- $P = 8, R = 1$ gives $2^8 = 256$ possible codes.
- `method="uniform"`: patterns with at most 2 bit transitions (edges, corners, spots) get their own code, all others share one, giving $P + 2 = 10$ **bins** and rotation invariance.
- The histogram of codes is the texture feature. It is robust to monotonic lighting changes because it uses only comparisons.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
from skimage import feature
image = load("data/image.jpg", cv2.IMREAD_GRAYSCALE)
radius = 1
n_points = 8 * radius
lbp = feature.local_binary_pattern(image, n_points, radius, method="uniform")
hist, _ = np.histogram(lbp.ravel(), bins=np.arange(0, n_points + 3), range=(0, n_points + 2))
hist = hist.astype("float") / (hist.sum() + 1e-6) # normalise: independent of image size
lbp_vis = (lbp / lbp.max() * 255).astype(np.uint8)
show([image, lbp_vis], ["Original", f"LBP uniform P={n_points}, R={radius}"])
plt.figure(figsize=(6, 3))
plt.bar(range(n_points + 2), hist)
plt.title(f"LBP histogram ({n_points + 2} bins)")
plt.xlabel("LBP pattern")
plt.ylabel("Frequency")
plt.show()
print("LBP feature vector:", hist.round(3))
cv2.imwrite("output/lbp_image.png", lbp_vis)
```

6.4 Colour histograms (1D, 2D, 3D)

Count pixels per colour bin. Global and rotation-invariant, but they ignore spatial layout.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/image.jpg")
bins1 = 256
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
for i, col in enumerate(("b", "g", "r")):
    h1 = cv2.calcHist([img], [i], None, [bins1], [0, 256]) # images, channels, mask, bins, range
    plt.plot(h1, color=col)
plt.title(f"1D BGR histograms, bins={bins1}")
plt.xlim([0, 256])

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
hb, sb = 30, 32
h2 = cv2.calcHist([hsv], [0, 1], None, [hb, sb], [0, 180, 0, 256]) # hue max is 180
plt.subplot(1, 2, 2)
plt.imshow(h2, interpolation="nearest") # a 2-D histogram, not a BGR image
plt.title(f"2D Hue-Saturation, {hb}x{sb} bins")
plt.xlabel("S bins")
plt.ylabel("H bins")
plt.show()

nb = 8
h3 = cv2.calcHist([img], [0, 1, 2], None, [nb] * 3, [0, 256] * 3)
feat3d = cv2.normalize(h3, None).flatten() # 8*8*8 = 512-D vector
concat = np.concatenate([cv2.calcHist([img], [c], None, [32], [0, 256]).ravel() for c in range(3)]) # 96-D
flipped = cv2.flip(img, 1)
h_f = cv2.normalize(cv2.calcHist([flipped], [0, 1, 2], None, [nb] * 3, [0, 256] * 3), None).flatten()
score = cv2.compareHist(feat3d, h_f, cv2.HISTCMP_CORREL) # 1.0 = identical
print(f"3D feature {feat3d.size}-D, concatenated {concat.size}-D, correlation with flipped = {score:.3f}")
show([img, flipped], ["Original", f"Flipped: histogram correlation={score:.3f}"])
cv2.imwrite("output/hist_flipped_compare.png", flipped)
```

The flipped image scores 1.0 because histograms ignore where pixels are. That is the main weakness of colour histograms.

6.5 HED (Histogram of Edge Directions)

In this course HED = **Histogram of Edge Directions**: edge map → gradient angle at each edge pixel → bin the angles (e.g. 8 bins over 360°) → count. It summarises the dominant edge orientations.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/image.jpg", cv2.IMREAD_GRAYSCALE)
smooth = cv2.GaussianBlur(image, (5, 5), 0)
lo, hi, nbins = 30, 70, 8
edges = cv2.Canny(smooth, lo, hi)
gx = cv2.Sobel(smooth, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(smooth, cv2.CV_64F, 0, 1, ksize=3)
angle = (np.degrees(np.arctan2(gy, gx)) + 360) % 360 # fix: arctan2 gives -180..180
hist, bin_edges = np.histogram(angle[edges > 0], bins=nbins, range=(0, 360)) # edge pixels only
show([image, edges], ["Original", f"Canny low={lo}, high={hi}"])
plt.figure(figsize=(6, 3))
plt.bar(bin_edges[:-1], hist, width=360 / nbins, align="edge")
plt.xticks(range(0, 361, 45))
plt.xlabel("Edge direction (deg)")
plt.title(f"HED, {nbins} bins")
plt.show()
print("HED feature vector:", hist)
cv2.imwrite("output/hed_edges.png", edges)
```

6.6 HIG (Histogram of Intensity Gradients)

Same gradient maths, but over **every pixel** (no edge map), commonly unsigned 0°–180° with 9 bins, weighted by magnitude and normalised. HOG is HIG computed per cell with block normalisation.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/image.jpg", cv2.IMREAD_GRAYSCALE)
dx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
dy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
magnitude, orientation = cv2.cartToPolar(dx, dy, angleInDegrees=True) # 0..360
orientation = orientation % 180 # unsigned
nbins = 9
hig, bins = np.histogram(orientation, bins=nbins, range=(0, 180), weights=magnitude)
hig = hig / hig.sum()
mag_vis = cv2.convertScaleAbs(magnitude)
show([image, mag_vis], ["Original", "Gradient magnitude"])
plt.figure(figsize=(6, 3))
plt.bar(bins[:-1], hig, width=180 / nbins, align="edge")
plt.title(f"HIG, {nbins} bins, magnitude-weighted")
plt.xlabel("Gradient orientation (deg)")
plt.show()
cv2.imwrite("output/hig_magnitude.png", mag_vis)
```

6.7 Texture energy and contrast

Over a $k \times k$ window W : energy is the sum of squared intensities; contrast is the local standard deviation.

$$E = \sum_{(i,j) \in W} I(i,j)^2, \quad C = \sqrt{\mathbb{E}_W[I^2] - (\mathbb{E}_W[I])^2}$$

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/image.jpg", cv2.IMREAD_GRAYSCALE)
f = image.astype(np.float32) # fix: uint8 ** 2 overflows
k = 5
mean = cv2.blur(f, (k, k)) # local E[I]
mean_sq = cv2.blur(f * f, (k, k)) # local E[I^2]
energy = mean_sq * k * k # local sum of squares
contrast = np.sqrt(np.maximum(mean_sq - mean ** 2, 0)) # local standard deviation
energy_vis = cv2.normalize(energy, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
contrast_vis = cv2.normalize(contrast, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
show([image, energy_vis, contrast_vis], ["Original", f"Texture energy, k={k}", f"Texture contrast (local std), k={k}"])
```

```
e_hist, _ = np.histogram(energy, bins=64)
c_hist, _ = np.histogram(contrast, bins=64)
plt.figure(figsize=(12, 3))
plt.subplot(121)
plt.plot(e_hist / e_hist.sum(), "b")
plt.title(f"Energy histogram, k={k}")
plt.subplot(122)
plt.plot(c_hist / c_hist.sum(), "r")
plt.title(f"Contrast histogram, k={k}")
plt.show()
cv2.imwrite("output/texture_contrast.png", contrast_vis)
```

Smooth textures (plastic, polished metal) → low contrast. Rough textures (bark, gravel, knitted fabric) → high contrast.

6.8 Lab 4 task: texture analysis for material classification

Model answer structure: describe two techniques, the textures each suits, then strengths and limitations.

	LBP	Texture energy / contrast
How it works	Binary code from neighbour-vs-centre comparisons; histogram of codes	Local sum of squares and local standard deviation in a sliding window
Best for	Fine micro-patterns: fabric weave, wood grain, brushed metal	Coarse roughness: polished vs rough metal, smooth vs rough wood
Strengths	Lighting-invariant, fast, rotation-invariant (uniform), compact 10-D vector	Very simple, intuitive, cheap with box filters
Limitations	Noise-sensitive at $R = 1$; misses large structure (use several radii)	Sensitive to brightness and scale; no direction information

Add HOG when shape matters, and colour histograms when materials differ in colour (colour fails across paint finishes).

The classifier below assigns the material whose feature vector is nearest: $\hat{y} = \arg \min_m \|\mathbf{f}_m - \mathbf{f}_{\text{test}}\|_2$.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
from skimage.feature import local_binary_pattern

def texture_vector(gray, P=8, R=1, k=5):
    lbp = local_binary_pattern(gray, P, R, "uniform")
    h, _ = np.histogram(lbp, bins=np.arange(P + 3), range=(0, P + 2))
    h = h / h.sum()
    f = gray.astype(np.float32)
    std = np.sqrt(np.maximum(cv2.blur(f * f, (k, k)) - cv2.blur(f, (k, k)) ** 2, 0))
    return np.concatenate([h, [std.mean() / 255, std.std() / 255]]) # 12-D

materials = ("wood", "metal", "fabric")
train_imgs = {}
for name in materials:
    train_imgs[name] = load(f"data/{name}.jpg", cv2.IMREAD_GRAYSCALE)
train = {name: texture_vector(im) for name, im in train_imgs.items()}
test_img = load("data/unknown.jpg", cv2.IMREAD_GRAYSCALE)
test = texture_vector(test_img)
dists = {n: np.linalg.norm(train[n] - test) for n in materials}
pred = min(dists, key=dists.get)
print("Distances:", {n: round(float(d), 3) for n, d in dists.items()}, "-> predicted:", pred)
show([train_imgs[n] for n in materials] + [test_img],
     [f"{n}: d={dists[n]:.3f}" for n in materials] + [f"Unknown -> {pred}"])
cv2.imwrite(f"output/material_prediction_{pred}.png", test_img)
```

6.9 Filtering and convolution

Slide a small kernel w over the image; each output pixel is the sum of kernel $\times$ neighbourhood products:

$$g(x, y) = \sum_{i=-a}^a \sum_{j=-b}^b w(i, j) f(x + i, y + j)$$

Strictly that is correlation; true convolution flips the kernel first. `cv2.filter2D` correlates; for symmetric kernels the result is identical.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/image.jpg")
k, sigma = 5, 1.0
box = cv2.filter2D(image, -1, np.ones((3, 3), np.float32) / 9) # mean of 9 neighbours; -1 = same depth
g1d = cv2.getGaussianKernel(k, sigma) # k x 1 column!
gauss = cv2.filter2D(image, -1, g1d @ g1d.T) # fix: outer product gives k x k
median = cv2.medianBlur(image, k) # best for salt-and-pepper noise
bilat = cv2.bilateralFilter(image, 9, 75, 75) # smooths, keeps edges
sharpen = cv2.filter2D(image, -1, np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]], np.float32))
emboss = cv2.filter2D(image, -1, np.array([[ -2, -1, 0], [-1, 1, 1], [0, 1, 2]], np.float32))
show([image, box, gauss, median, bilat, sharpen, emboss],
     ["Original", "Box 3x3", f"Gaussian k={k}, sigma={sigma}", f"Median k={k}", "Bilateral d=9, 75/75",
      "Sharpen", "Emboss"], cols=4)
cv2.imwrite("output/filter_sharpen.png", sharpen)
```

`cv2.blur(image, (3, 3))` equals the box filter; `cv2.GaussianBlur(image, (k, k), sigma)` equals the Gaussian. The manual passes `getGaussianKernel` straight to `filter2D`; that kernel is $k \times 1$, so it blurs vertically only.

Kernel	Matrix	Effect
Box 3×3	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	Average, blur
Gaussian 3×3	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	Smooth blur, less blocky
Sobel G_x	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	Vertical edges (horizontal change)
Sobel G_y	$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$	Horizontal edges (vertical change)
Prewitt G_x	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	Like Sobel, no centre weighting
Laplacian	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	Second derivative, all directions
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	Identity plus Laplacian boost

6.10 Convolution types: standard, valid, same, strided

For input size n , kernel k , padding p and stride s :

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1, \quad \text{valid } (p = 0) : n - k + 1, \quad \text{same } (p = \frac{k-1}{2}, s = 1) : n$$

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
gray = load("data/image.jpg", cv2.IMREAD_GRAYSCALE).astype(np.float32)
kernel = np.array([[1, 0, -1], [2, 0, -2], [1, 0, -1]], np.float32)
k = kernel.shape[0]
p = k // 2
standard = cv2.filter2D(gray, -1, kernel) # default border = REFLECT_101
same_zero = cv2.filter2D(gray, -1, kernel, borderType=cv2.BORDER_CONSTANT) # true zero padding
valid = same_zero[p:-p, p:-p] # true valid output: (H-k+1, W-k+1)
strided = valid[::2, ::2] # stride 2 (filter2D has no stride argument)
print("input", gray.shape, "| valid", valid.shape, "| stride 2", strided.shape)

def conv2d(img, ker, stride=1, pad=0):
    """Convolution from scratch: flip kernel, zero-pad, slide with a stride."""
```

```

ker = np.flipud(np.fliplr(ker))
img = np.pad(img, pad, mode="constant")
kh, kw = ker.shape
oh = (img.shape[0] - kh) // stride + 1
ow = (img.shape[1] - kw) // stride + 1
out = np.zeros((oh, ow), np.float32)
for i in range(oh):
    for j in range(ow):
        r, c = i * stride, j * stride
        out[i, j] = np.sum(img[r:r + kh, c:c + kw] * ker)
return out

manual = conv2d(gray[:100, :100], kernel, stride=2, pad=1) # small crop: Python loops are slow
views = [cv2.convertScaleAbs(x) for x in (standard, same_zero, valid, strided, manual)]
show(views, ["Standard (reflect)", "Same (zero pad)", f"Valid {valid.shape}", f"Stride 2 {strided.shape}",
            f"Manual s=2, p=1 {manual.shape}"], cols=5)
cv2.imwrite("output/conv_valid.png", views[2])

```

`cv2.convertScaleAbs(result)` takes the absolute value and clips to uint8 for display.

6.11 Edge detection operators

Method	Order	Notes
Sobel	1st derivative	G_x, G_y with centre weight 2; simple; thick edges; noise-sensitive
Scharr	1st	More accurate rotation behaviour for 3×3
Prewitt	1st	Equal weights; no OpenCV function, use <code>filter2D</code>
Laplacian	2nd	Rotation-invariant; very noise-sensitive; double edges
LoG (Marr-Hildreth)	2nd	Gaussian blur, then Laplacian; edges at zero crossings
Canny	1st + logic	Blur, gradient, NMS, hysteresis; thin connected edges
CNN-based	learned	Adapts to the domain (e.g. Holistically-Nested Edge Detection)

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}, \quad \text{LoG} = \nabla^2 (G_\sigma * I)$$

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
gray = load("data/image.jpg", cv2.IMREAD_GRAYSCALE)
ks, T = 3, 100
sx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=ks) # CV_64F keeps negative gradients
sy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=ks)
mag = np.sqrt(sx ** 2 + sy ** 2)
gf = gray.astype(np.float32)
px = cv2.filter2D(gf, -1, np.array([[ -1, 0, 1]] * 3, np.float32)) # Prewitt x
py = cv2.filter2D(gf, -1, np.array([[ -1] * 3, [0] * 3, [1] * 3], np.float32)) # Prewitt y
prewitt = cv2.convertScaleAbs(np.sqrt(px ** 2 + py ** 2))
lap = cv2.convertScaleAbs(cv2.Laplacian(gray, cv2.CV_64F, ksize=ks))
log = cv2.convertScaleAbs(cv2.Laplacian(cv2.GaussianBlur(gray, (5, 5), 1.4), cv2.CV_64F))
edges_bin = (mag > T).astype(np.uint8) * 255 # threshold the magnitude
show([gray, cv2.convertScaleAbs(sx), cv2.convertScaleAbs(sy), cv2.convertScaleAbs(mag), prewitt, lap, log,
      edges_bin],
      ["Gray", f"Sobel X k={ks}", f"Sobel Y k={ks}", "Sobel magnitude", "Prewitt", f"Laplacian k={ks}", "LoG",
       sigma=1.4", f"|G| > {T}"], cols=4)
cv2.imwrite("output/edges_sobel_binary.png", edges_bin)

```

Why `CV_64F`? A dark-to-bright edge gives a positive gradient, bright-to-dark a negative one. In uint8 the negatives clip to 0 and half the edges vanish.

6.12 Canny in four stages

- Gaussian smoothing:** remove noise so it does not create false edges. Larger σ gives fewer, smoother edges and loses fine detail.
- Gradient:** Sobel $G_x, G_y \rightarrow$ magnitude $|\nabla I|$ and direction θ per pixel.

3. **Non-maximum suppression:** keep a pixel only if it is the peak along its gradient direction (compare with the two neighbours across the edge). Thick ridges become 1-pixel lines.
4. **Hysteresis:** above T_{high} = strong edge (kept). Below T_{low} = discarded. In between = weak; kept **only if connected** to a strong edge.

$$|\nabla I| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan \frac{G_y}{G_x}, \quad T_{\text{high}} \approx 2T_{\text{low}} \text{ to } 3T_{\text{low}}$$

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
image = load("data/image.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
sigma, lo, hi = 1.4, 50, 150
blurred = cv2.GaussianBlur(gray, (5, 5), sigma)
edges = cv2.Canny(blurred, lo, hi)
edges_l2 = cv2.Canny(blurred, lo, hi, apertureSize=3, L2gradient=True) # exact sqrt magnitude
v = np.median(blurred) # automatic thresholds from the median
a_lo, a_hi = int(max(0, 0.67 * v)), int(min(255, 1.33 * v))
auto = cv2.Canny(blurred, a_lo, a_hi)
show([image, blurred, edges, edges_l2, auto],
     ["Original", f"Gaussian 5x5, sigma={sigma}", f"Canny low={lo}, high={hi}", f"Canny {lo}/{hi}, L2", f"Auto Canny {a_lo}/{a_hi}"], cols=5)
cv2.imwrite("output/canny_edges.png", edges)
```

Sobel vs Canny (manual's comparison): Sobel is one gradient step with thick, noisy edges. Canny is a full pipeline with thin, connected, noise-robust edges.

7. Lab 05: Image segmentation (all 10 tasks)

Segmentation splits an image into non-overlapping regions that share colour, intensity or texture. Every Lab 5 task must show the original and the result, record and justify parameters, save the output, and use no prebuilt segmentation library or deep model.

7.1 Methods at a glance

Method	Assumption about the image	Key parameter	Fails when
Global threshold	One intensity separates object from background everywhere	T	Lighting varies across the image
Adaptive threshold	Each small neighbourhood has roughly even lighting	blockSize, C	Block too small (noise) or too big (acts global)
Otsu	Histogram is bimodal (two peaks)	none (automatic)	One class tiny, or three or more classes
Colour (HSV <code>inRange</code>)	Target has a distinctive hue	H, S, V bounds	Hue shared with background; shadows; highlights
Canny (hysteresis)	Boundaries are sharp intensity changes	$T_{\text{low}}, T_{\text{high}}$	Low contrast or texture; gives edges, not regions
Region growing	Target is connected and roughly uniform	seed, tolerance	Leaks through weak boundaries; seed-dependent
Watershed	Objects are blobs; touching ones have a "neck"	distance-transform threshold	Too low merges, too high over-splits or misses
K-means	Regions differ in colour; K roughly known	K	Same colour, different objects; random init

Segmentation maths in the manual: thresholding labels a pixel object when $I(x, y) > T$; K-means minimises the within-cluster sum of squared distances J ; graph methods cut a pixel graph at minimum total edge weight.

$$S(x, y) = \begin{cases} \text{object} & I(x, y) > T \\ \text{background} & \text{otherwise} \end{cases} \quad J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Every block below assumes the Section 1.4 master utilities have run (`load` , `show` , `output/` folder).

7.2 Task 1: uneven lighting, global vs adaptive (sudoku.png)

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
gray = load("data/sudoku.png", cv2.IMREAD_GRAYSCALE)
T_values = [60, 100, 140]
block, C = 11, 2
globs = [cv2.threshold(gray, t, 255, cv2.THRESH_BINARY)[1] for t in T_values]
adaptive = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, block, C)
show([gray] + globs + [adaptive],
     ["Original"] + [f"Global T={t}" for t in T_values] + [f"Adaptive Gaussian block={block}, C={C}"], cols=5)
h, w = gray.shape
quads = {"TL": gray[:h // 2, :w // 2], "TR": gray[:h // 2, w // 2:], "BL": gray[h // 2:, :w // 2], "BR": gray[h // 2:, w // 2:]}
for name, q in quads.items():
    print(f"{name} mean brightness = {q.mean():.1f}") # darkest quadrant = where global fails
cv2.imwrite("output/task1_adaptive.png", adaptive)
for t, g in zip(T_values, globs):
    cv2.imwrite(f"output/task1_global_T{t}.png", g)
```

Parameters: $T = 60, 100, 140$ bracket the middle of the histogram. A block of 11 is about one character stroke region; $C = 2$ removes faint background noise.

Answer: a single T compares every pixel with one number. Where illumination is high, paper exceeds T and stays white, but in shadowed regions dark text and background both fall below the bright-side level. A low T loses characters in shadow; a high T turns whole shadowed areas black. The quadrant with the lowest mean is where global thresholding fails. Adaptive thresholding computes a separate $T(x, y)$ for each pixel from its own neighbourhood, so it follows the lighting gradient.

7.3 Task 2: adaptive blockSize, C and method grid

The threshold at each pixel is the local mean (or Gaussian-weighted mean) of a $b \times b$ window minus C :

$$T(x, y) = \text{mean}_{b \times b}(x, y) - C$$

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
gray = load("data/sudoku.png", cv2.IMREAD_GRAYSCALE)
blocks, Cs = [5, 15, 51], [2, 5, 10]
methods = {"Mean": cv2.ADAPTIVE_THRESH_MEAN_C, "Gaussian": cv2.ADAPTIVE_THRESH_GAUSSIAN_C}
results, titles = [], []
for mname, m in methods.items():
    for b in blocks:
        for c in Cs:
            res = cv2.adaptiveThreshold(gray, 255, m, cv2.THRESH_BINARY, b, c)
            results.append(res)
            titles.append(f"{mname} block={b}, C={c}")
            cv2.imwrite(f"output/task2_{mname}_b{b}_C{c}.png", res)
show(results, titles, cols=3, size=3)
```

Question	Answer
Neighbourhood too small ($b = 5$)	The window sits inside a stroke or on flat paper, so tiny noise counts as detail: speckle and hollow or broken characters.
Neighbourhood too large ($b = 51$)	The window spans several lighting levels and behaves like global thresholding: shadows return, thin lines thicken or merge.
Increasing C	Lowers every local threshold, so fewer pixels count as dark: less background noise, but thin or faint strokes break. Lower C = more noise.

Question	Answer
Cleanest combination	Typically Gaussian with $b = 11$ to 15 and $C = 2$ to 5 for sudoku.png. Pick from your grid and say why.
Better method	Usually Gaussian: nearby pixels weigh more, so edges are smoother with less blocky noise. Mean is slightly faster.

Rules: blockSize must be **odd and** > 1 ; the input must be 8-bit single channel.

7.4 Task 3: Otsu on coins, with contrast changes

Otsu tries every T and picks the one that maximises the between-class variance (equivalently minimises within-class variance), where ω_0, ω_1 are class probabilities and μ_0, μ_1 class means:

$$T^* = \arg \max_T \sigma_B^2(T) = \omega_0(T) \omega_1(T) [\mu_0(T) - \mu_1(T)]^2$$

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/coins.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
variants = {
    "original": gray,
    "contrast x1.5": cv2.convertScaleAbs(gray, alpha=1.5, beta=0), # new = clip(1.5 * old)
    "brighter +40": cv2.convertScaleAbs(gray, alpha=1.0, beta=40),
    "blurred 5x5": cv2.GaussianBlur(gray, (5, 5), 0),
    "equalized": cv2.equalizeHist(gray),
}
masks, titles = [], []
for name, g in variants.items():
    T, mask = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU) # the 0 is ignored
    masks.append(mask)
    titles.append(f"{name}: Otsu T={T:.0f}")
    cv2.imwrite(f"output/task3_otsu_{name.replace(' ', '_')}.png", mask)
T0, _ = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
plt.figure(figsize=(6, 3))
plt.hist(gray.ravel(), 256, (0, 256), color="gray")
plt.axvline(T0, color="r")
plt.title(f"Original histogram, Otsu T={T0:.0f}")
plt.show()
show([img] + masks, ["Original"] + titles, cols=3)
```

Answer: Otsu reads the histogram and finds the valley between the two peaks automatically, so the same code adapts to each new image. When contrast or brightness change, the peaks move and Otsu's T moves with them (record the printed values), while a fixed manual T breaks. It relies on a bimodal histogram; blur first if noise fills the valley. Objects darker than the background? Use `THRESH_BINARY_INV + THRESH_OTSU` to make them white.

7.5 Task 4: HSV colour segmentation (too-restrictive vs final mask)

HSV separates colour (Hue) from strength (Saturation) and brightness (Value), so a shadow changes V but barely changes H . OpenCV ranges: $H \in [0, 179]$, $S, V \in [0, 255]$.

Colour	H range (OpenCV)
Red	0–10 and 170–179 (wraps around)
Orange	10–22
Yellow	22–35
Green	35–85
Cyan / light blue	85–100
Blue	100–130
Purple / magenta	130–170

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/smarties.png")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
print("HSV at (x=100, y=80):", hsv[80, 100]) # probe a pixel you know is the target

lo_r, hi_r = (25, 200, 200), (30, 255, 255) # too restrictive
lo_f, hi_f = (20, 100, 100), (35, 255, 255) # final yellow range
restrictive = cv2.inRange(hsv, np.array(lo_r), np.array(hi_r))
final = cv2.inRange(hsv, np.array(lo_f), np.array(hi_f))
k = np.ones((5, 5), np.uint8)
final = cv2.morphologyEx(final, cv2.MORPH_OPEN, k) # remove specks
final = cv2.morphologyEx(final, cv2.MORPH_CLOSE, k) # fill small holes
extracted = cv2.bitwise_and(img, mask=final)
red = cv2.inRange(hsv, (0, 100, 80), (10, 255, 255)) | cv2.inRange(hsv, (170, 100, 80), (179, 255, 255)) # red
= 2 ranges
show([img, restrictive, final, extracted, red],
     ["Original", f"Restrictive {lo_r}-{hi_r}", f"Final {lo_f}-{hi_f} + open/close", "Extracted", "Red: two hue
ranges"], cols=5)
cv2.imwrite("output/task4_mask.png", final)
cv2.imwrite("output/task4_extracted.png", extracted)
```

Why the restrictive mask fails: a real object is not one colour. Curved surfaces have shading (lower V) and specular highlights (lower S , higher V), and the camera adds noise. A narrow H band with $S, V \geq 200$ keeps only the brightest, most saturated pixels, so the mask is patchy with holes and missing edges. Lowering the S and V bounds to about 100 accepts the shaded parts while H still rejects other colours.

7.6 Task 5: Canny with three threshold pairs

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/smarties.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 1.4)
pairs = [(50, 100), (50, 200), (150, 300)] # 1 & 2: high rises, low fixed; 3: both high
res = [cv2.Canny(blur, lo, hi) for lo, hi in pairs]
for (lo, hi), e in zip(pairs, res):
    print(f"low={lo}, high={hi}: edge pixels = {np.count_nonzero(e)}")
    cv2.imwrite(f"output/task5_canny_{lo}_{hi}.png", e)

lo, hi = 50, 150 # strong vs weak candidates before linking (approximate magnitude)
mag = np.hypot(cv2.Sobel(blur, cv2.CV_64F, 1, 0), cv2.Sobel(blur, cv2.CV_64F, 0, 1))
vis = np.zeros((*gray.shape, 3), np.uint8)
vis[mag >= lo & (mag < hi)] = (0, 255, 255) # weak: yellow (BGR)
vis[mag >= hi] = (0, 0, 255) # strong: red
show([gray] + res + [vis], ["Original"] + [f"Canny low={lo_}, high={hi_}" for lo_, hi_ in pairs] + [f"Strong >=
{hi} (red), weak {lo}-{hi} (yellow)"], cols=5)
cv2.imwrite("output/task5_strong_weak.png", vis)
```

Term	Where you see it
Strong edges	Outer outlines of objects; survive all three pairs.
Weak edges	Faint inner shading lines; present at 50/100, gone at 150/300.
Missing edges	Low-contrast boundaries (two similar colours touching) vanish when T_{high} rises.
Unwanted edges	Texture, noise and highlights appear when T_{low} is small.

Explanation: T_{high} decides which pixels **start** an edge: raising it ($50/100 \rightarrow 50/200$) removes outlines that had no strong pixel. T_{low} decides how far an edge can **extend**: lowering it lets edges continue through faint parts (better continuity, more noise). Raising both (150/300) keeps only the strongest boundaries.

7.7 Task 6: region growing on a brain MRI (from scratch)

A pixel joins the region if it is connected to the region and close in intensity to the seed (x_s, y_s) :

$$R = \{ (x, y) \text{ connected to } (x_s, y_s) : |I(x, y) - I(x_s, y_s)| < \tau \}$$

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
from collections import deque

def region_growing(img, seed, thresh, connectivity=4):
    """seed = (x, y). Adds neighbours whose |I - I_seed| < thresh. Returns a uint8 mask."""
    h, w = img.shape
    mask = np.zeros((h, w), np.uint8)
    visited = np.zeros((h, w), bool)
    sx, sy = seed
    seed_val = int(img[sy, sx]) # NumPy is [row=y, col=x]
    q = deque([(sx, sy)])
    visited[sy, sx] = True
    nbrs = [(1, 0), (-1, 0), (0, 1), (0, -1)]
    if connectivity == 8:
        nbrs += [(1, 1), (1, -1), (-1, 1), (-1, -1)]
    while q:
        x, y = q.popleft()
        if abs(int(img[y, x]) - seed_val) < thresh:
            mask[y, x] = 255
            for dx, dy in nbrs:
                nx, ny = x + dx, y + dy
                if 0 <= nx < w and 0 <= ny < h and not visited[ny, nx]:
                    visited[ny, nx] = True
                    q.append((nx, ny))
    return mask

mri = load("data/brain_mri.png", cv2.IMREAD_GRAYSCALE)
mri = cv2.GaussianBlur(mri, (5, 5), 0) # smoothing reduces leaks through noise
h, w = mri.shape
seeds = [(w // 2, h // 2), (w // 3, h // 2)] # change to points inside your target
thresholds = [10, 25, 50]
results, titles = [mri], ["MRI (blurred)"]
for s in seeds:
    for t in thresholds:
        m = region_growing(mri, s, t)
        results.append(m)
        titles.append(f"seed={s}, tau={t}: {np.count_nonzero(m)} px")
        cv2.imwrite(f"output/task6_seed{s[0]}_{s[1]}_{t}.png", m)
overlay = cv2.cvtColor(mri, cv2.COLOR_GRAY2BGR)
overlay[results[2] > 0] = (0, 0, 255) # seed 1, tau=25 in red
show(results + [overlay], titles + [f"Overlay seed={seeds[0]}, tau={thresholds[1]}"], cols=4)
cv2.imwrite("output/task6_overlay.png", overlay)

```

Pick a seed by plotting with axes on and reading the (x, y) of the target, or seed on the brightest pixel with `np.unravel_index(np.argmax(mri), mri.shape)` (which returns (y, x)).

Why the seed changes everything: the region is "pixels connected to the seed whose intensity is close to the seed's intensity". A different seed has a different reference value (so a different set of pixels qualifies) and starts in a different connected area (so it can only spread through its own neighbours). Seeding in grey matter versus white matter, or in a lesion, yields entirely different masks. τ too small $\rightarrow$ the region stops early; too large $\rightarrow$ it leaks across weak boundaries.

Variant: compare with the **running region mean** instead of the seed value for smoother growth.

7.8 Task 7: marker-based watershed for touching coins

The distance transform gives each foreground pixel its distance to the nearest background pixel, $D(p) = \min_{q \in \text{bg}} \|p - q\|_2$, so each coin centre becomes a peak even where coins touch.

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/coins.jpg")
gray = cv2.GaussianBlur(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), (5, 5), 0) # 1 preprocessing
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU) # 2 threshold
border = np.r_[thresh[0], thresh[-1], thresh[:, 0], thresh[:, -1]]
if border.mean() > 127:
    thresh = cv2.bitwise_not(thresh) # coins must be WHITE
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2) # 3 noise removal

```

```

sure_bg = cv2.dilate(opening, kernel, iterations=3) # 4 sure background
dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5) # 5 distance transform
frac = 0.5
_, sure_fg = cv2.threshold(dist, frac * dist.max(), 255, 0) # 6 sure foreground
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg) # 7 unknown ring
n, markers = cv2.connectedComponents(sure_fg) # 8 labels 0..n-1
markers = markers + 1 # background becomes 1, not 0
markers[unknown == 255] = 0 # 0 = let watershed decide
markers = cv2.watershed(img, markers) # 9 needs a 3-channel 8-bit image
out = img.copy()
out[markers == -1] = (0, 0, 255) # 10 boundaries in RED (BGR)
count = len([l for l in np.unique(markers) if l > 1])
dist_vis = cv2.normalize(dist, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
print("foreground markers:", n - 1, "| separated coins:", count)
show([img, thresh, sure_bg, dist_vis, sure_fg, unknown, out],
      ["Original", "Otsu threshold", "Sure background", "Distance transform", f"Sure FG > {frac} x max",
       "Unknown", f"Watershed: {count} coins"], cols=4)
cv2.imwrite("output/task7_watershed.png", out)

```

How it works: thresholding the distance peaks gives one seed per coin (sure foreground). Dilating gives a region surely outside the coins (sure background). The ring between them is unknown. Watershed floods from the labelled seeds and builds a dam (label -1) where floods from two seeds meet: the dividing line between touching coins.

After `watershed`: -1 = boundary, 1 = background, $2 \dots n$ = individual objects.

7.9 Task 8: tuning the distance-transform threshold

```

import cv2, numpy as np, matplotlib.pyplot as plt, pandas as pd # load(), show() from Section 1.4

def watershed_count(img, frac):
    gray = cv2.GaussianBlur(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), (5, 5), 0)
    _, th = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
    if np.r_[th[0], th[-1], th[:, 0], th[:, -1]].mean() > 127:
        th = cv2.bitwise_not(th)
    k = np.ones((3, 3), np.uint8)
    op = cv2.morphologyEx(th, cv2.MORPH_OPEN, k, iterations=2)
    bg = cv2.dilate(op, k, iterations=3)
    d = cv2.distanceTransform(op, cv2.DIST_L2, 5)
    _, fg = cv2.threshold(d, frac * d.max(), 255, 0)
    fg = np.uint8(fg)
    n, mk = cv2.connectedComponents(fg)
    mk = mk + 1
    mk[cv2.subtract(bg, fg) == 255] = 0
    mk = cv2.watershed(img.copy(), mk)
    out = img.copy()
    out[mk == -1] = (0, 0, 255)
    return n - 1, len([l for l in np.unique(mk) if l > 1]), out

img = load("data/coins.jpg")
fracs = [0.1, 0.3, 0.5, 0.7, 0.9]
rows, outs = [], []
for f in fracs:
    nfg, nreg, out = watershed_count(img, f)
    rows.append((f, nfg, nreg))
    outs.append(out)
    cv2.imwrite(f"output/task8_dist_{f}.png", out)
show(outs, [f"threshold={f} x max: {a} markers, {b} regions" for f, a, b in rows], cols=3)
print(pd.DataFrame(rows, columns=["Distance threshold (x max)", "Foreground markers",
    "Regions"]).to_string(index=False))

```

Fill the "observed result" column from your own pictures. The typical pattern:

Threshold	Effect
Low (0.1–0.2)	Sure-foreground blobs still touch, so touching coins share one marker: merged .
Middle (0.4–0.6)	One compact marker per coin: best separation . Report this range.

Threshold	Effect
High (0.8–0.9)	Small or less-round coins lose their peak entirely: missed objects ; irregular shapes may split into two peaks: over-segmentation .

`cv2.distanceTransform` arguments: `DIST_L2` = Euclidean distance; mask size 5 (3 is faster and coarser; `DIST_MASK_PRECISE` is exact).

7.10 Task 9: K-means colour segmentation (K = 2, 4, 6)

Each pixel is a 3-D colour vector $\mathbf{x}_i$. K-means alternates two steps until the centres stop moving: assign each pixel to its nearest centre, then move each centre to the mean of its pixels, $\mu_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/dog.jpeg")
pixels = np.float32(img.reshape(-1, 3)) # (N, 3) feature vectors; float32 required
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2) # max 100 iterations or move < 0.2
attempts = 10
results, titles = [img], ["Original"]
for K in (2, 4, 6):
    compactness, labels, centers = cv2.kmeans(pixels, K, None, criteria, attempts, cv2.KMEANS_PP_CENTERS)
    seg = np.uint8(centers[labels.flatten()]).reshape(img.shape) # each pixel -> its centre colour
    shares = np.bincount(labels.flatten(), minlength=K) / labels.size
    print(f"K={K}: compactness={compactness:.3e}")
    for c, p in zip(np.uint8(centers), shares):
        print(f"centre BGR {tuple(int(v) for v in c)} covers {p:.1%}")
    results.append(seg)
    titles.append(f"K={K}, attempts={attempts}, compactness={compactness:.2e}")
    cv2.imwrite(f"output/task9_kmeans_K{K}.png", seg)
show(results, titles)
```

- $K = 2$: light vs dark (foreground vs background silhouette); heavy simplification.
- $K = 4$: main regions appear (fur, background, shadows, highlights); moderate simplification.
- $K = 6$: close to the original with posterised flat colours; least simplification.
- **Compactness** J falls as K rises; the "elbow" where it stops dropping sharply suggests a good K .
- To group by colour regardless of brightness, cluster in Lab or HSV. Add pixel coordinates as extra features for spatially compact clusters.

7.11 Task 10: segmentation engineer, three methods on one image

Choose `smarties.png` (coloured, touching objects, uneven background): each method shows a clear strength and weakness on it.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/smarties.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Method 1: Otsu
T, m1 = cv2.threshold(cv2.GaussianBlur(gray, (5, 5), 0), 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
# Method 2: HSV saturation mask (colourful candies vs pale background)
s_min = 80
m2 = cv2.inRange(hsv, (0, s_min, 50), (179, 255, 255))
m2 = cv2.morphologyEx(m2, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))
# Method 3: K-means
K = 4
px = np.float32(img.reshape(-1, 3))
_, lab, cen = cv2.kmeans(px, K, None, (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2), 10, cv2.KMEANS_PP_CENTERS)
m3 = np.uint8(cen[lab.flatten()]).reshape(img.shape)

for name, m in [("Otsu", m1), ("HSV", m2)]:
    n, _ = cv2.connectedComponents(m)
    print(f"{name}: {n - 1} connected objects")
show([img, m1, m2, m3], ["Original", f"Method 1: Otsu T={T:.0f}", f"Method 2: HSV S>={s_min}", f"Method 3: K-means K={K}"])
```

```
cv2.imwrite("output/task10_otsu.png", m1)
cv2.imwrite("output/task10_hsv.png", m2)
cv2.imwrite("output/task10_kmeans.png", m3)
```

Write this table from **your** outputs (the cells show the expected pattern):

	Otsu	HSV colour	K-means
Assumption	Bimodal grey histogram	Objects are more saturated than background	Regions differ in colour; K known
Succeeds on	Dark or strongly coloured candies	All coloured candies, incl. ones similar in grey	Groups candies by colour family
Fails on	Yellow or light candies (grey close to background), shadows	Pale or low-saturation items, specular highlights	Shadows become their own cluster; no object separation
Most sensitive parameter	Pre-blur size	S lower bound	K

Conclusion template: "HSV produced the most meaningful regions because the defining property of this image is colour saturation, not brightness. Several candies have the same grey level as the background, which breaks Otsu's bimodal assumption, while K-means groups by colour family but cannot separate object from shadow. To count touching candies, the HSV mask would feed into watershed (Task 7)."

7.12 Morphology cheat (used throughout)

Op	Code	Effect
Erode	<code>cv2.erode(m, k, iterations=1)</code>	Shrinks white, removes specks
Dilate	<code>cv2.dilate(m, k, iterations=1)</code>	Grows white, fills gaps
Opening	<code>cv2.morphologyEx(m, cv2.MORPH_OPEN, k)</code>	Erode then dilate: removes small noise
Closing	<code>cv2.morphologyEx(m, cv2.MORPH_CLOSE, k)</code>	Dilate then erode: fills small holes
Gradient	<code>cv2.morphologyEx(m, cv2.MORPH_GRADIENT, k)</code>	Outline of shapes

In set notation, opening is $A \circ B = (A \ominus B) \oplus B$ and closing is $A \bullet B = (A \oplus B) \ominus B$. Round kernel: `k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))`.

8. Lab 06: Wavelets, Hough transform, SIFT & applications

The sixth manual (titled "Lab 05: Wavelets, Boundary Detection, Hough Transform & SIFT") pairs with the Lab 06 task sheet: wavelets for signals, Hough voting for lines and circles, and SIFT matching with RANSAC homography for recognition and stitching.

8.1 Wavelet transforms

A wavelet is a short, localised wave. Sliding it across a signal at different **scales** a and **shifts** b shows which frequencies occur **when**, which a Fourier transform cannot (Fourier gives frequencies for the whole signal at once). That makes wavelets ideal for non-stationary signals: spikes, bursts, sudden faults.

$$W(a, b) = \frac{1}{\sqrt{|a|}} \int_{-\infty}^{\infty} x(t) \psi^* \left(\frac{t-b}{a} \right) dt$$

Type	What it does	Families	Uses
CWT (continuous)	Every scale and shift; redundant, detailed time-frequency map	Morlet (<code>morl</code>), Mexican hat	Seismic, oscillation analysis
DWT (discrete)	Splits into approximation cA (low frequency) + detail cD (high frequency), then repeats on cA	Haar, Daubechies (<code>db4</code>), Symlets	Compression, denoising, features

Type	What it does	Families	Uses
WPT (packet)	Splits both cA and cD at each level	Same as DWT	Complex frequency content, ML features
Inverse (IDWT)	Rebuilds the signal from coefficients		Reconstruction after denoising

Denoising idea: noise lives in small detail coefficients; real structure lives in large ones. Shrink small details toward zero (soft threshold) and reconstruct. The universal threshold, with σ estimated from the finest details d_1 :

$$\lambda = \sigma\sqrt{2\ln N}, \quad \sigma = \frac{\text{median}(|d_1|)}{0.6745}, \quad \text{soft}(d) = \text{sign}(d) \max(|d| - \lambda, 0)$$

```
import cv2, numpy as np, matplotlib.pyplot as plt, pywt # load(), show() from Section 1.4
np.random.seed(0)
t = np.linspace(0, 1, 1024)
signal = np.sin(2 * np.pi * 5 * t) + 0.3 * np.random.randn(t.size)
wavelet, level = "db4", 4
cA, cD = pywt.dwt(signal, wavelet) # one level: approximation, detail
coeffs = pywt.wavedec(signal, wavelet, level=level) # [cA4, cD4, cD3, cD2, cD1]
sigma = np.median(np.abs(coeffs[-1])) / 0.6745
lam = sigma * np.sqrt(2 * np.log(signal.size))
den = [coeffs[0]] + [pywt.threshold(c, lam, mode="soft") for c in coeffs[1:]]
denoised = pywt.waverec(den, wavelet)[:signal.size] # inverse DWT (trim padding)
cwt_mat, _ = pywt.cwt(signal, np.arange(1, 64), "morl") # CWT scalogram (scales x time)
wp = pywt.WaveletPacket(signal, wavelet, maxlevel=3) # WPT
print("WPT level-3 bands:", [n.path for n in wp.get_level(3, "natural")]) # 'aaa', 'aad', ...

fig, ax = plt.subplots(3, 1, figsize=(12, 8))
ax[0].plot(signal, label="noisy")
ax[0].plot(denoised, label="denoised")
ax[0].legend()
ax[0].set_title(f"DWT denoising: {wavelet}, level={level}, lambda={lam:.2f}")
ax[1].imshow(np.abs(cwt_mat), aspect="auto", cmap="jet") # a coefficient map, not a BGR image
ax[1].set_title("CWT scalogram (Morlet), scales 1-63")
ax[2].plot(cD)
ax[2].set_title(f"Level-1 detail coefficients ({wavelet})")
plt.tight_layout()
plt.savefig("output/wavelet_signal.png")
plt.show()

# 2-D DWT on an image: LL (approximation) and LH, HL, HH (details)
gray = load("data/image.jpg", cv2.IMREAD_GRAYSCALE).astype(np.float32)
LL, (LH, HL, HH) = pywt.dwt2(gray, "haar")
bands = [cv2.normalize(np.abs(b), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8) for b in (LL, LH, HL, HH)]
show(bands, ["LL (approximation)", "LH (horizontal detail)", "HL (vertical detail)", "HH (diagonal detail)"])
cv2.imwrite("output/dwt2_LL.png", bands[0])
```

Audio denoising (the manual's figure): read with `scipy.io.wavfile.read`, add noise, apply the same `wavedec` → threshold → `waverec`, and plot the three waveforms.

8.2 Boundary detection recap

Sobel, Canny and LoG code is in Sections 6.11 and 6.12. MCQ points from this manual:

- Sobel uses two 3×3 kernels; magnitude = edge strength, direction = edge orientation. G_x responds to vertical edges, G_y to horizontal ones.
- Worked example: a 3×3 patch with every row $[10, 10, 200]$ gives $G_x = (200 + 400 + 200) - (10 + 20 + 10) = 760$; a uniform patch gives 0.
- `CV_64F` keeps negative gradients (the direction of change).
- First-derivative **peaks** mark edges (Sobel, Canny). Second-derivative **zero crossings** mark edge locations (LoG). The LoG pipeline is Gaussian blur → Laplacian → zero-crossing detection.

8.3 Hough transform for lines

Every edge pixel votes for all lines that could pass through it; lines with many votes are real. The manual starts from $y = mx + b$ (parameter space (m, b)), but vertical lines have infinite slope, so OpenCV uses the normal form:

$$\rho = x \cos \theta + y \sin \theta, \quad \theta \in [0, \pi)$$

Each pixel becomes a sinusoid in (ρ, θ) space; where many sinusoids cross is a line. Steps: edge detection → voting into an accumulator → peak detection → back-transform to image lines.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150, apertureSize=3)
votes = 150
lines = cv2.HoughLines(edges, 1, np.pi / 180, votes) # rho res 1 px, theta res 1 deg
std = img.copy()
if lines is not None: # returns None when nothing is found
    for rho, theta in lines.reshape(-1, 2):
        a, b = np.cos(theta), np.sin(theta)
        x0, y0 = a * rho, b * rho
        p1 = (int(x0 - 1000 * b), int(y0 + 1000 * a))
        p2 = (int(x0 + 1000 * b), int(y0 - 1000 * a))
        cv2.line(std, p1, p2, (0, 0, 255), 2)
pt, min_len, gap = 80, 50, 10
segs = cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=pt, minLineLength=min_len, maxLineGap=gap)
prob = img.copy()
if segs is not None:
    for x1, y1, x2, y2 in segs.reshape(-1, 4):
        cv2.line(prob, (x1, y1), (x2, y2), (0, 255, 0), 2)
n_std = 0 if lines is None else len(lines)
n_seg = 0 if segs is None else len(segs)
show([img, edges, std, prob], ["Original", "Canny 50/150", f"HoughLines votes>={votes}: {n_std}",
                                f"HoughLinesP t={pt}, minLen={min_len}, gap={gap}: {n_seg}"])
cv2.imwrite("output/hough_lines.png", prob)
```

Tuning: more lines → lower `threshold` or `minLineLength`; join broken lines → raise `maxLineGap`. To keep near-horizontal or near-vertical segments, compute $\varphi = \arctan 2(y_2 - y_1, x_2 - x_1)$ and keep $|\varphi| < 10^\circ$ or $||\varphi| - 90^\circ| < 10^\circ$.

8.4 Hough transform for circles

$$(x - a)^2 + (y - b)^2 = r^2 \quad (3\text{-D parameter space } (a, b, r))$$

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/coins.jpg")
gray = cv2.medianBlur(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), 5) # blur is essential: fewer false circles
dp, min_dist, p1, p2, r_min, r_max = 1.2, 40, 100, 30, 15, 80
circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=dp, minDist=min_dist,
                           param1=p1, param2=p2, minRadius=r_min, maxRadius=r_max)
# dp: accumulator resolution; minDist: between centres; param1: Canny high threshold;
# param2: accumulator votes (LOWER = more circles, more false ones)
out = img.copy()
n = 0
if circles is not None:
    for x, y, r in np.round(circles[0]).astype(int):
        cv2.circle(out, (x, y), r, (0, 255, 0), 2)
        cv2.circle(out, (x, y), 2, (0, 0, 255), 3)
    n = circles.shape[1]
show([img, out], ["Original", f"HoughCircles dp={dp}, minDist={min_dist}, p1={p1}, p2={p2}, r={r_min}-{r_max}: {n}"])
cv2.imwrite("output/hough_circles.png", out)
```

8.5 SIFT: the full matching pipeline

Stage	What happens
1 Scale-space extrema	Blur at increasing σ and subtract neighbours (DoG). Search across scales so features are found whether large or small.
2 Keypoint localisation	A candidate must beat all 26 neighbours (8 same scale + 9 above + 9 below). Refine with a 3-D quadratic fit; drop low-contrast and edge points.
3 Orientation assignment	Dominant gradient direction in the neighbourhood (36-bin histogram) gives rotation invariance.
4 Descriptor	A 16×16 patch $\rightarrow 4 \times 4$ sub-regions $\times$ 8-bin orientation histograms = 128 numbers , robust to lighting.
5 Matching	Euclidean distance between descriptors (BFMatcher or FLANN) plus Lowe's ratio test.
6 Outlier rejection	RANSAC keeps only geometrically consistent matches.
7 Homography	The inliers give H for recognition or stitching.

$$\text{DoG}(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma), \quad \text{keep a match if } \frac{d_1}{d_2} < 0.75$$

Ratio test intuition: if the best match d_1 is barely better than the second best d_2 , the feature is ambiguous (repeated texture), so drop it. RANSAC then repeatedly fits **H** to 4 random matches and keeps the model with the most inliers, where an inlier satisfies $\|\mathbf{H}\mathbf{p}_i - \mathbf{p}'_i\| < 5$ px.

Define these two helpers once; Sections 8.7 and 8.10 reuse them.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4

def sift_match(img1, img2, ratio=0.75, min_matches=10, use_flann=False):
    """img1 = reference/object, img2 = scene. Returns H (img1 -> img2), good matches, inlier mask, keypoints."""
    sift = cv2.SIFT_create() # optional: SIFT_create(nfeatures=1000)
    g1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY) if img1.ndim == 3 else img1
    g2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY) if img2.ndim == 3 else img2
    kp1, des1 = sift.detectAndCompute(g1, None)
    kp2, des2 = sift.detectAndCompute(g2, None)
    if des1 is None or des2 is None or len(kp2) < 2:
        return None, [], None, kp1, kp2
    if use_flann:
        matcher = cv2.FlannBasedMatcher(dict(algorithm=1, trees=5), dict(checks=50)) # KD-tree
    else:
        matcher = cv2.BFMatcher(cv2.NORM_L2) # L2 for SIFT; NORM_HAMMING for ORB
    knn = matcher.knnMatch(des1, des2, k=2) # two nearest neighbours each
    good = [p[0] for p in knn if len(p) == 2 and p[0].distance < ratio * p[1].distance] # Lowe's test
    if len(good) < min_matches:
        return None, good, None, kp1, kp2
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0) # 5 px reprojection tolerance
    return H, good, mask, kp1, kp2

def draw_box(scene, ref, H, text=None, color=(0, 255, 0)):
    """Project the reference image's corners into the scene and outline them."""
    h, w = ref.shape[:2]
    corners = np.float32([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]]).reshape(-1, 1, 2)
    box = cv2.perspectiveTransform(corners, H)
    out = cv2.polylines(scene.copy(), [np.int32(box)], True, color, 3)
    if text:
        cv2.putText(out, text, tuple(int(v) for v in box[0, 0]), cv2.FONT_HERSHEY_SIMPLEX, 0.9, color, 2)
    return out

# Keypoints only (manual 5.7)
image = load("data/image.jpg")
kp, des = cv2.SIFT_create().detectAndCompute(cv2.cvtColor(image, cv2.COLOR_BGR2GRAY), None)
rich = cv2.drawKeypoints(image, kp, None, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS) # circle = scale,
line = orientation
print("Keypoints:", len(kp), "| descriptor shape:", des.shape) # (N, 128)
```

```
# Matching with inliers drawn
ref = load("data/object.jpg")
scene = load("data/scene.jpg")
ratio = 0.75
H, good, mask, kp1, kp2 = sift_match(ref, scene, ratio=ratio)
if H is not None:
    vis = cv2.drawMatches(ref, kp1, draw_box(scene, ref, H), kp2, good, None,
                          matchesMask=mask.ravel().tolist(), flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
    show([rich, vis], [f"SIFT keypoints: {len(kp)}", f"ratio={ratio}: {int(mask.sum())} inliers / {len(good)} good"], cols=1, size=6)
    cv2.imwrite("output/sift_matches.png", vis)
else:
    print("Object not found:", len(good), "good matches")
    show([rich], [f"SIFT keypoints: {len(kp)}"])
cv2.imwrite("output/sift_keypoints.png", rich)
```

ORB is the free, faster alternative (binary descriptor): `cv2.ORB_create(1000)` with `cv2.BFMatcher(cv2.NORM_HAMMING)`. SURF is patented and missing from standard builds.

8.6 Task: computer screen detection (Hough lines) + document scanner

Screens are bright or dark rectangles bounded by long straight edges. Strategy: Canny → Hough segments for evidence of edges → contour polygons with 4 corners and a screen-like aspect ratio → ON/OFF by mean brightness.

`HoughLinesP` is probabilistic and can skip one side of a rectangle when several screens share a straight top edge, so the Canny edges are OR-ed back in. In testing, that took detection from 2 of 3 screens to 3 of 3.

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4

def order_points(pts):
    """Return 4 points ordered TL, TR, BR, BL."""
    pts = pts.reshape(4, 2).astype(np.float32)
    s, d = pts.sum(axis=1), np.diff(pts, axis=1).ravel() # x+y and y-x
    return np.float32([pts[np.argmin(s)], pts[np.argmax(d)], pts[np.argmax(s)], pts[np.argmin(d)]])

def four_point_warp(img, pts):
    rect = order_points(pts)
    tl, tr, br, bl = rect
    W = int(max(np.linalg.norm(br - bl), np.linalg.norm(tr - tl)))
    Hh = int(max(np.linalg.norm(tr - br), np.linalg.norm(tl - bl)))
    M = cv2.getPerspectiveTransform(rect, np.float32([[0, 0], [W - 1, 0], [W - 1, Hh - 1], [0, Hh - 1]]))
    return cv2.warpPerspective(img, M, (W, Hh))

img = load("data/lab.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(cv2.GaussianBlur(gray, (5, 5), 0), 50, 150)
segs = cv2.HoughLinesP(edges, 1, np.pi / 180, 80, minLineLength=60, maxLineGap=10)
line_mask = np.zeros_like(gray)
if segs is not None:
    for x1, y1, x2, y2 in segs.reshape(-1, 4):
        ang = abs(np.degrees(np.arctan2(y2 - y1, x2 - x1)))
        if ang < 10 or abs(ang - 90) < 10 or ang > 170: # keep horizontal + vertical only
            cv2.line(line_mask, (x1, y1), (x2, y2), 255, 3)
line_mask = cv2.bitwise_or(line_mask, cv2.dilate(edges, np.ones((3, 3), np.uint8))) # fill gaps HoughLinesP skipped
line_mask = cv2.morphologyEx(line_mask, cv2.MORPH_CLOSE, np.ones((7, 7), np.uint8)) # join corners
cnts, _ = cv2.findContours(line_mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
out, screens, accepted, warped = img.copy(), 0, [], None
on_thresh, EXPECTED = 90, 6 # brightness for ON; screens in a full row
for c in sorted(cnts, key=cv2.contourArea, reverse=True):
    if cv2.contourArea(c) < 0.005 * gray.size:
        continue
    approx = cv2.approxPolyDP(c, 0.03 * cv2.arcLength(c, True), True)
    x, y, w, h = cv2.boundingRect(approx)
    cx, cy = x + w // 2, y + h // 2
    if len(approx) == 4 and 1.1 < w / h < 2.2 and not any(ax <= cx <= ax + aw and ay <= cy <= ay + ah for ax, ay, aw, ah in accepted):
        accepted.append((x, y, w, h)) # skip bezel/display duplicates
        screens += 1
        status = "ON" if gray[y:y + h, x:x + w].mean() > on_thresh else "OFF"
```

```

cv2.drawContours(out, [approx], -1, (0, 255, 0) if status == "ON" else (0, 0, 255), 3)
cv2.putText(out, f"S{screens} {status}", (x, max(y - 8, 15)), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 0), 2)

if warped is None:
    warped = four_point_warp(img, approx) # document-scanner style flattening of the first screen
print(f"screens found: {screens}; missing: {max(EXPECTED - screens, 0)}")
show([img, edges, line_mask, out], ["Original", "Canny 50/150", "Hough H/V lines (closed)", f"{screens} screens, ON if mean>{on_thresh}"])
cv2.imwrite("output/screens_detected.png", out)
if warped is not None:
    cv2.imwrite("output/screen_flattened.png", warped)

```

Document scanner: run the same contour step on `edges`, take the **largest** 4-point contour, then `four_point_warp(img, approx)`.

8.7 Task: asset tracking with SIFT (inventory)

Store one reference photo per asset; an asset is present in a scene if enough RANSAC inliers match. Needs `sift_match()` and `draw_box()` from Section 8.5.

```

import cv2, numpy as np, matplotlib.pyplot as plt, pandas as pd # load(), show() from 1.4; sift_match(), draw_box() from 8.5
ref_paths = {"Monitor-01": "data/monitor.jpg", "Keyboard-01": "data/keyboard.jpg", "CPU-01": "data/cpu.jpg"}
refs = {}
for name, path in ref_paths.items():
    refs[name] = load(path)
scene = load("data/lab_desk.jpg")
min_inliers = 12
inventory, out = {}, scene.copy()
for name, ref in refs.items():
    H, good, mask, _, _ = sift_match(ref, scene, ratio=0.75, min_matches=min_inliers)
    inliers = 0 if mask is None else int(mask.sum())
    found = H is not None and inliers >= min_inliers
    inventory[name] = {"present": found, "good": len(good), "inliers": inliers}
    if found:
        out = draw_box(out, ref, H, text=name)
print(pd.DataFrame(inventory).T)
show([scene, out], ["Scene", f"Detected assets (inliers >= {min_inliers})"])
cv2.imwrite("output/asset_inventory.png", out)

```

Why SIFT: identical-looking models are told apart by stickers, logos and asset tags (local texture), and detection survives distance (scale), camera angle (rotation) and partial occlusion.

8.8 Task: sensor anomaly detection with wavelets

The residual is $r_t = x_t - \hat{x}_t$ (signal minus denoised). A sample is an anomaly when $|r_t| > k \sigma_r$.

```

import cv2, numpy as np, matplotlib.pyplot as plt, pywt
np.random.seed(0)
N = 1000
sensor = np.random.normal(0, 1, N) # sample sensor stream
sensor[[150, 420, 700, 880]] += [6, -5, 7, -6] # injected faults
# real data: sensor = pd.read_csv("data/sensor.csv")["value"].to_numpy()
wavelet, level, k = "db4", 4, 3.0 # tested: k=3 caught all 4 injected faults + 1 false alarm; k=2 flagged 44 points
coeffs = pywt.wavedec(sensor, wavelet, level=level)
sigma = np.median(np.abs(coeffs[-1])) / 0.6745
lam = sigma * np.sqrt(2 * np.log(N))
denoised = pywt.waverec([coeffs[0]] + [pywt.threshold(c, lam, "soft") for c in coeffs[1:]], wavelet)[:N]
residuals = sensor - denoised
anoms = np.where(np.abs(residuals) > k * residuals.std())[0]
print(f"{len(anoms)} anomalies at indices: {anoms}")

fig, ax = plt.subplots(2, 1, figsize=(12, 7))
ax[0].plot(sensor, label="Sensor Data")
ax[0].plot(denoised, "--", label="Denoised Signal")
ax[0].set_title(f"Sensor Data and Denoised Signal ({wavelet}, level={level})")
ax[0].legend()

```

```

ax[1].plot(residuals, "r", label="Residuals")
ax[1].scatter(anoms, residuals[anoms], c="g", zorder=3, label="Anomalies")
ax[1].set_title(f"Residuals and Detected Anomalies ( $|r| > \{k\} \sigma$ )")
ax[1].legend()
plt.tight_layout()
fig.canvas.draw()
plot_img = cv2.cvtColor(np.asarray(fig.canvas.buffer_rgba()), cv2.COLOR_RGBA2BGR) # figure -> BGR image
plt.show()
cv2.imwrite("output/sensor_anomalies.png", plot_img)

```

Real-time version: keep a sliding window (e.g. the last 256 samples), recompute for each new sample, and flag the newest point if its residual exceeds $k\sigma_r$. Explanation: denoising removes ordinary noise; whatever the clean model cannot explain (the residual) and is unusually large is abnormal.

8.9 Task: object recognition in video (SIFT)

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
ref = load("data/object.jpg")
sift = cv2.SIFT_create()
kp_r, des_r = sift.detectAndCompute(cv2.cvtColor(ref, cv2.COLOR_BGR2GRAY), None) # compute ONCE
bf = cv2.BFMatcher(cv2.NORM_L2)
h, w = ref.shape[:2]
corners = np.float32([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]]).reshape(-1, 1, 2)
ratio, min_good, min_inliers = 0.75, 10, 8
cap = cv2.VideoCapture("data/test_video.mp4")
if not cap.isOpened():
    raise FileNotFoundError("Error: could not open video data/test_video.mp4")
last, found_frames, frame_no = None, 0, 0
while True:
    ok, frame = cap.read()
    if not ok:
        break
    frame_no += 1
    kp_f, des_f = sift.detectAndCompute(cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY), None)
    text, color = "NOT FOUND", (0, 0, 255)
    if des_f is not None and len(kp_f) >= 2:
        pairs = [p for p in bf.knnMatch(des_r, des_f, k=2) if len(p) == 2]
        good = [m for m, n in pairs if m.distance < ratio * n.distance]
        if len(good) >= min_good:
            src = np.float32([kp_r[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
            dst = np.float32([kp_f[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
            H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
            if H is not None and mask.sum() >= min_inliers:
                cv2.polylines(frame, [np.int32(cv2.perspectiveTransform(corners, H))], True, (0, 255, 0), 3)
                text, color = f"FOUND ({int(mask.sum())} inliers)", (0, 255, 0)
                found_frames += 1
    cv2.putText(frame, text, (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)
    cv2.imshow("Recognition", frame)
    last = frame
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
cap.release()
cv2.destroyAllWindows()
print(f"Object found in {found_frames}/{frame_no} frames")
if last is not None:
    show([ref, last], ["Reference", f"Last frame, ratio={ratio}, min inliers={min_inliers}"])
    cv2.imwrite("output/recognition_last_frame.png", last)

```

Speed-up: resize frames to 640 px width, or process every 2nd frame. For test images instead of video, loop over `sorted(Path("data").glob("test_*.jpg"))` with `load()`, `sift_match()` and `draw_box()`.

8.10 Task: panorama stitching

The right image is warped into the left image's frame with the homography: $\mathbf{p}_{\text{left}} \sim \mathbf{H} \mathbf{p}_{\text{right}}$. Needs `sift_match()` from Section 8.5.

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from 1.4; sift_match() from 8.5
left = load("data/left.jpg") # overlapping photos taken while panning
right = load("data/right.jpg")
H, good, mask, _, _ = sift_match(right, left) # maps RIGHT into LEFT's frame
if H is None:
    raise RuntimeError("Not enough overlap or matches to stitch")
h1, w1 = left.shape[:2]
h2, w2 = right.shape[:2]
pano = cv2.warpPerspective(right, H, (w1 + w2, max(h1, h2))) # wide canvas
pano[0:h1, 0:w1] = left # paste the reference image on top
g = cv2.cvtColor(pano, cv2.COLOR_BGR2GRAY)
x, y, w, h = cv2.boundingRect(cv2.findNonZero(g)) # crop the black border
pano = pano[y:y + h, x:x + w]
show([left, right, pano], ["Left", "Right", f"Panorama: {int(mask.sum())} inliers / {len(good)} matches"],
cols=3)
cv2.imwrite("output/panorama.png", pano)

status, pano2 = cv2.Stitcher_create(cv2.Stitcher_PANORAMA).stitch([left, right]) # built-in alternative
if status == cv2.Stitcher_OK:
    show([pano2], [cv2.Stitcher(blending + exposure)])
    cv2.imwrite("output/panorama_stitcher.png", pano2)
else:
    print("Stitcher failed, status", status)

```

More than two images: stitch left to right in pairs, treating the growing panorama as `left`. Aim for about 30% overlap. A visible seam can be softened by blending the overlap with `cv2.addWeighted` instead of pasting.

8.11 Task: lane detection with Hough lines

Each segment gives a slope and intercept, $m = \frac{y_2 - y_1}{x_2 - x_1}$ and $b = y_1 - mx_1$. Because y points down, the left lane has $m < 0$ and the right lane $m > 0$. Averaging each side gives one line, drawn between two rows using $x = \frac{y-b}{m}$.

```

import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4

def lane_lines(frame, top=0.6, min_slope=0.4):
    h, w = frame.shape[:2]
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(cv2.GaussianBlur(gray, (5, 5), 0), 50, 150)
    roi = np.array([(int(0.1 * w), h), (int(0.45 * w), int(top * h)),
                    (int(0.55 * w), int(top * h)), (int(0.95 * w), h)], np.int32) # road trapezoid
    mask = np.zeros_like(edges)
    cv2.fillPoly(mask, roi, 255)
    masked = cv2.bitwise_and(edges, mask)
    segs = cv2.HoughLinesP(masked, 2, np.pi / 180, 50, minLineLength=40, maxLineGap=100)
    left, right = [], []
    if segs is not None:
        for x1, y1, x2, y2 in segs.reshape(-1, 4):
            if x2 == x1:
                continue # vertical: infinite slope
            m = (y2 - y1) / (x2 - x1)
            b = y1 - m * x1
            if abs(m) < min_slope:
                continue # near-horizontal noise
            (left if m < 0 else right).append((m, b))
    overlay = np.zeros_like(frame)
    y_bot, y_top = h, int(top * h)
    for group in (left, right):
        if group:
            m, b = np.mean(group, axis=0) # average: one line per side
            cv2.line(overlay, (int((y_bot - b) / m), y_bot), (int((y_top - b) / m), y_top), (0, 0, 255), 10)
    return cv2.addWeighted(frame, 0.8, overlay, 1.0, 0), masked, len(left), len(right)

road = load("data/road.jpg")
result, masked, nl, nr = lane_lines(road)
show([road, masked, result], ["Road", "Canny inside ROI trapezoid", f"Lanes: {nl} left / {nr} right segments averaged"])
cv2.imwrite("output/lanes.png", result)

```


For video, call `lane_lines(frame)` inside the read loop (Section 2.6). For yellow lanes on grey roads, OR a yellow HSV mask with a white mask before Canny.

8.12 Task: coin detection and counting (Hough circles)

```
import cv2, numpy as np, matplotlib.pyplot as plt # load(), show() from Section 1.4
img = load("data/coins.jpg")
gray = cv2.medianBlur(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), 5)
p2, min_dist, r_min, r_max = 25, 30, 15, 90 # tested: 25 found all 4 coins; 30 found 1 on a low-contrast image;
tune 20-35
circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1.2, minDist=min_dist,
                           param1=100, param2=p2, minRadius=r_min, maxRadius=r_max)
out = img.copy()
count = 0
if circles is not None:
    circles = np.round(circles[0]).astype(int)
    count = len(circles)
    small = circles[:, 2] < np.median(circles[:, 2]) # crude size classes
    for (x, y, r), s in zip(circles, small):
        cv2.circle(out, (x, y), r, (0, 255, 0) if s else (255, 0, 0), 3)
        cv2.putText(out, f"r{r}", (x - 15, y + 5), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 2)
cv2.putText(out, f"Coins: {count}", (10, 35), cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0, 0, 255), 3)

_, bw = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU) # cross-check with contours
if np.r_[bw[0], bw[-1]].mean() > 127:
    bw = cv2.bitwise_not(bw)
cnts, _ = cv2.findContours(bw, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
n_cnt = sum(cv2.contourArea(c) > 300 for c in cnts)
print(f"Hough: {count} | contours > 300 px: {n_cnt}")
show([img, bw, out], ["Original", f"Otsu binary: {n_cnt} blobs", f"Hough param2={p2}, r={r_min}-{r_max}: {count} coins"])
cv2.imwrite("output/coins_counted.png", out)
```

Missing coins → lower `param2` or widen the radius range. Extra circles → raise `param2`, and set `minDist` near the smallest coin diameter. Touching coins defeat contour counting but not Hough (each circle votes separately); watershed (Section 7.8) is the region-based alternative. To total the value, map radius bands to denominations: $\text{total} = \sum_i v(r_i)$.

8.13 Task: smart security zone (boundary detection in real-time video)

An object intrudes when its overlap with the zone mask Z exceeds a fraction of its area: $\frac{|O \cap Z|}{|O|} > 0.2$.

```
import cv2, numpy as np, matplotlib.pyplot as plt # show() from Section 1.4
cap = cv2.VideoCapture(0) # webcam; or "data/cctv.mp4"
if not cap.isOpened():
    raise FileNotFoundError("Error: could not open camera/video source")
bg = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=25, detectShadows=True)
zone = np.array([[200, 150], [450, 150], [450, 400], [200, 400]], np.int32) # restricted area (x, y)
min_area, overlap_frac = 800, 0.2
last, alarm_frames = None, 0
while True:
    ok, frame = cap.read()
    if not ok:
        break
    fg = bg.apply(frame) # 255 = moving, 127 = shadow, 0 = background
    _, fg = cv2.threshold(fg, 200, 255, cv2.THRESH_BINARY) # drop shadows
    fg = cv2.morphologyEx(fg, cv2.MORPH_OPEN, np.ones((3, 3), np.uint8))
    fg = cv2.dilate(fg, np.ones((5, 5), np.uint8), iterations=2)
    zone_mask = np.zeros(fg.shape, np.uint8)
    cv2.fillPoly(zone_mask, [zone], 255)
    cnts, _ = cv2.findContours(fg, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    alarm = False
    for c in cnts:
        if cv2.contourArea(c) < min_area:
            continue # ignore small motion
        x, y, w, h = cv2.boundingRect(c)
        obj = np.zeros_like(fg)
        cv2.drawContours(obj, [c], -1, 255, -1)
```



```

        inside = cv2.countNonZero(cv2.bitwise_and(obj, zone_mask)) > overlap_frac * cv2.contourArea(c)
        alarm |= inside
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 0, 255) if inside else (0, 255, 0), 2)
        cv2.polylines(frame, [zone], True, (0, 0, 255) if alarm else (255, 255, 0), 2)
    if alarm:
        alarm_frames += 1
        cv2.putText(frame, "ALARM: UNAUTHORIZED OBJECT", (10, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255),
3)
        cv2.imwrite("output/security_alarm.png", frame) # evidence snapshot
    cv2.imshow("Security", frame)
    cv2.imshow("Foreground", fg)
    last = frame
    if cv2.waitKey(30) & 0xFF == ord("q"):
        break
cap.release()
cv2.destroyAllWindows()
print("Frames with alarm:", alarm_frames)
if last is not None:
    show([last], [f"Last frame: min area={min_area}, overlap>{overlap_frac}"])
    cv2.imwrite("output/security_last_frame.png", last)

```

- Simpler alternative to MOG2: frame differencing against an empty-scene reference, `diff = cv2.absdiff(gray, ref_gray)`, then threshold at about 25.
- Point-in-zone test for a centroid: `cv2.pointPolygonTest(zone, (cx, cy), False)` returns 1 inside, 0 on the edge, -1 outside.
- Beep on Windows: `import winsound; winsound.Beep(1000, 200)`.
- MOG2 needs a few dozen frames to learn the background, and objects that stop moving slowly fade into it. For "objects placed and left", compare against a fixed empty-scene reference instead.

9. MCQ rapid-fire & function signatures

Q1 is MCQs: these are the facts most likely to be tested, then the exact signatures to copy under pressure.

9.1 Facts to know cold

Topic	Fact
Default colour order	<code>cv2.imread</code> returns BGR , uint8, shape $(H, W, 3)$
Bad path	<code>imread</code> returns <code>None</code> , no exception
<code>resize</code> dsize	(w, h) ; <code>.shape</code> is (h, w)
Gaussian kernel	Odd size; $\sigma = 0$ means derived from size
<code>threshold</code> return	<code>(retval, image)</code> ; with Otsu, <code>retval</code> is the chosen T
<code>equalizeHist</code> input	8-bit, single channel only
<code>cv2.add</code> vs <code>+</code>	Saturate at 255 vs wrap modulo 256
<code>addWeighted</code>	$\alpha \text{src}_1 + \beta \text{src}_2 + \gamma$; same size and type
Rotation angle sign	Positive = counter-clockwise on screen
<code>warpAffine</code> matrix	2×3 ; <code>warpPerspective</code> takes 3×3
Points needed	Affine 3 pairs, perspective 4 pairs
Degrees of freedom	Translation 2, rigid 3, similarity 4, affine 6, projective 8
Affine preserves	Collinearity, parallelism, ratios of distances along a line
Projective breaks	Parallelism (lines converge to vanishing points)
Linear can't translate	$\mathbf{A}\mathbf{0} = \mathbf{0}$: the origin is fixed
Homogeneous point	$(x, y) \rightarrow (x, y, 1)$; 2-D needs 3×3 , 3-D needs 4×4
Point vs geometric	Photometric changes values; geometric changes positions

Topic	Fact
Log transform	$s = c \log(1 + r)$ expands darks, compresses brights
Gamma	$s = r^\gamma$: $\gamma < 1$ brightens, $\gamma > 1$ darkens
Histogram equalization	$s_k = (L - 1) \text{CDF}(r_k)$; flattens the histogram; global
CLAHE	Local tile equalization with a clip limit to stop noise blow-up
HSV hue range in OpenCV	0 to 179 (not 360)
Red in HSV	Two ranges: 0–10 and 170–179
HOG defaults	9 bins, 8×8 cells, 2×2 blocks, 0° – 180° ; 3780-D for 64×128
LBP uniform bins	$P + 2$ (10 for $P = 8$)
SIFT descriptor	$4 \times 4 \times 8 = 128$ -D; extrema beat 26 neighbours in DoG
Lowe's ratio	Keep if $d_1/d_2 < 0.75$ (0.7–0.8 common)
RANSAC	Robust model fit that rejects outlier matches
BFMatcher norm	<code>NORM_L2</code> for SIFT, <code>NORM_HAMMING</code> for ORB
Hough line space	(ρ, θ) with $\rho = x \cos \theta + y \sin \theta$
Hough circle space	3-D: (a, b, r)
<code>HoughCircles</code> param2	Lower = more (and more false) circles
Canny stages	Blur → gradient → NMS → hysteresis
Canny thresholds	$> T_{\text{high}}$ strong; between = weak, kept if connected; $< T_{\text{low}}$ dropped
LoG edges	Zero crossings of the second derivative $\nabla^2 I$
Sobel worked example	G_x on rows $[10, 10, 200] = 760$
Otsu assumption	Bimodal histogram; maximises between-class variance σ_B^2
Adaptive blockSize	Odd and > 1
Watershed markers	–1 boundary, 1 background, 0 = unknown before the run
K-means input	float32, shape $(N, \text{features})$
Opening / closing	$A \circ B$ erode → dilate removes specks; $A \bullet B$ dilate → erode fills holes
DWT output	Approximation cA (low) + detail cD (high) coefficients
CWT wavelet	Morlet for oscillations
Hardest DIP step	Segmentation
Enhancement vs restoration	Subjective look vs objective undo of a degradation

9.2 Signatures

Function	Signature (key args)
imread	<code>cv2.imread(path, cv2.IMREAD_COLOR / IMREAD_GRAYSCALE / IMREAD_UNCHANGED)</code>
imwrite	<code>cv2.imwrite(path, img) -> bool</code>
cvtColor	<code>cv2.cvtColor(img, cv2.COLOR_BGR2RGB / BGR2GRAY / BGR2HSV / GRAY2BGR)</code>
resize	<code>cv2.resize(img, (w, h))</code> or <code>cv2.resize(img, None, fx=, fy=, interpolation=)</code>
GaussianBlur	<code>cv2.GaussianBlur(img, (k, k), sigmaX)</code>
medianBlur	<code>cv2.medianBlur(img, k)</code> (odd int)
blur	<code>cv2.blur(img, (k, k))</code>
filter2D	<code>cv2.filter2D(img, ddepth=-1, kernel, borderType=)</code>
threshold	<code>ret, dst = cv2.threshold(gray, T, maxval, cv2.THRESH_BINARY [+ cv2.THRESH_OTSU])</code>
adaptiveThreshold	<code>cv2.adaptiveThreshold(gray, 255, ADAPTIVE_THRESH_MEAN_C / GAUSSIAN_C, THRESH_BINARY, blockSize, C)</code>

Function	Signature (key args)
inRange	<code>cv2.inRange(hsv, lower, upper)</code> → 0/255 mask
equalizeHist	<code>cv2.equalizeHist(gray)</code>
CLAHE	<code>cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(gray)</code>
calcHist	<code>cv2.calcHist([img], [ch], mask, [bins], [lo, hi])</code>
applyColorMap	<code>cv2.applyColorMap(gray8, cv2.COLORMAP_JET)</code>
LUT	<code>cv2.LUT(img, table256_uint8)</code>
add / addWeighted	<code>cv2.add(a, b)</code> , <code>cv2.addWeighted(a, alpha, b, beta, gamma)</code>
bitwise	<code>cv2.bitwise_and(a, b, mask=m)</code> , <code>bitwise_or</code> , <code>bitwise_xor</code> , <code>bitwise_not(a)</code>
getRotationMatrix2D	<code>cv2.getRotationMatrix2D((cx, cy), angle_deg, scale)</code>
warpAffine	<code>cv2.warpAffine(img, M2x3, (w, h), borderValue=)</code>
getAffineTransform	<code>cv2.getAffineTransform(src3x2_f32, dst3x2_f32)</code>
getPerspectiveTransform	<code>cv2.getPerspectiveTransform(src4x2_f32, dst4x2_f32)</code>
warpPerspective	<code>cv2.warpPerspective(img, H, (w, h))</code>
perspectiveTransform	<code>cv2.perspectiveTransform(pts_Nx1x2_f32, H)</code>
Sobel / Scharr	<code>cv2.Sobel(img, cv2.CV_64F, dx, dy, ksize=3)</code> , <code>cv2.Scharr(img, cv2.CV_64F, dx, dy)</code>
Laplacian	<code>cv2.Laplacian(img, cv2.CV_64F, ksize=)</code>
Canny	<code>cv2.Canny(img, low, high, apertureSize=3, L2gradient=False)</code>
HoughLines	<code>cv2.HoughLines(edges, rho, theta, threshold)</code> → (N, 1, 2) or None; iterate <code>lines.reshape(-1, 2)</code>
HoughLinesP	<code>cv2.HoughLinesP(edges, rho, theta, threshold, minLineLength=, maxLineGap=)</code> → (N, 1, 4) in OpenCV 4, (N, 4) in OpenCV 5; iterate <code>segs.reshape(-1, 4)</code>
HoughCircles	<code>cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp, minDist, param1=, param2=, minRadius=, maxRadius=)</code> → (1, N, 3)
findContours	<code>cnts, hier = cv2.findContours(bin, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)</code>
Contour measures	<code>contourArea</code> , <code>arcLength(c, True)</code> , <code>boundingRect</code> , <code>minAreaRect</code> , <code>convexHull</code> , <code>moments</code> , <code>approxPolyDP(c, eps, True)</code>
morphologyEx	<code>cv2.morphologyEx(m, cv2.MORPH_OPEN / CLOSE / GRADIENT, kernel, iterations=)</code>
distanceTransform	<code>cv2.distanceTransform(bin, cv2.DIST_L2, 5)</code>
connectedComponents	<code>n, labels = cv2.connectedComponents(bin)</code> (n includes background)
watershed	<code>markers = cv2.watershed(img_bgr, markers_int32)</code>
kmeans	<code>compact, labels, centers = cv2.kmeans(data_f32, K, None, criteria, attempts, cv2.KMEANS_PP_CENTERS)</code>
SIFT	<code>kp, des = cv2.SIFT_create().detectAndCompute(gray, None)</code>
Matchers	<code>cv2.BFMatcher(cv2.NORM_L2).knnMatch(d1, d2, k=2)</code> , <code>cv2.FlannBasedMatcher(dict(algorithm=1, trees=5), dict(checks=50))</code>
findHomography	<code>H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)</code>
Drawing	<code>cv2.line/rectangle(img, p1, p2, color, thick)</code> , <code>cv2.circle(img, c, r, color, thick)</code> , <code>cv2.putText(img, text, org, font, scale, color, thick)</code> , <code>cv2.fillPoly(img, [pts], color)</code> , <code>cv2.polylines(img, [pts], True, color, thick)</code>
Video	<code>cv2.VideoCapture(path or 0)</code> , <code>cap.read()</code> → (ok, frame), <code>cv2.VideoWriter(path, cv2.VideoWriter_fourcc(*"mp4v"), fps, (w, h))</code>
scikit-image	<code>hog(img, orientations=9, pixels_per_cell=(8, 8), cells_per_block=(2, 2), visualize=True)</code> , <code>local_binary_pattern(img, P, R, method="uniform")</code>
PyWavelets	<code>pywt.dwt(x, "db4")</code> , <code>wavedec(x, "db4", level=)</code> , <code>waverec(c, "db4")</code> , <code>threshold(c, t, "soft")</code> , <code>cwt(x, scales, "morl")</code> , <code>dwt2(img, "haar")</code>

9.3 Common errors and instant fixes

Error message (short)	Cause	Fix
<code>!_src.empty()</code> in <code>cvtColor</code>	Image failed to load (<code>None</code>)	Use <code>load()</code> ; check the path
Sizes of input arguments do not match	Different sizes or channels in <code>add/bitwise/hstack</code>	<code>match()</code> from Section 1.4
<code>(-215) ... depth == CV_8U</code>	Float image passed to <code>equalizeHist</code> , <code>threshold</code> or <code>Canny</code>	<code>np.clip(x, 0, 255).astype(np.uint8)</code>
<code>'NoneType' ... not subscriptable</code>	Hough found nothing	<code>if lines is not None:</code>
not enough values to unpack (expected 3, got 2)	<code>findContours</code> return count (OpenCV 3 vs 4)	<code>cnts = cv2.findContours(...)[-2]</code>
<code>blockSize % 2 == 1 && blockSize > 1</code>	Even <code>blockSize</code> in <code>adaptiveThreshold</code>	Use odd values
<code>ksize.width > 0 && ksize.width % 2 == 1</code>	Even Gaussian kernel	Use odd values
Colours look blue-ish in Matplotlib	BGR shown as RGB	Use <code>show()</code> , or <code>cv2.cvtColor(img, cv2.COLOR_BGR2RGB)</code>
Black or nearly black output	Float range wrong, or gamma applied on <code>[0, 255]</code> instead of <code>[0, 1]</code>	Normalise, then clip
no attribute <code>'SIFT_create'</code>	OpenCV older than 4.4	<code>pip install opencv-contrib-python</code> (before the exam)